시스템 정의서(프로젝트 요약서)

팀 번호(1 - 09)

<table>
<tr><td rowspan="2">작품명
(주제)</td><td colspan="3">(국문) AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)</td></tr>
<tr><td colspan="3">(영문) AI/MCP Integrated Relational Database Management System (RuSQL)</td></tr>
<tr><td rowspan="3">책 임 자</td><td>성 명</td><td colspan="2">김규현</td></tr>
<tr><td>소 속</td><td colspan="2">충북대학교 소프트웨어학과</td></tr>
<tr><td>학 번</td><td colspan="2">2021041017</td></tr>
<tr><td>개발기간</td><td colspan="3">2026년 3월 2일 ~ 2026년 12월 31일</td></tr>
<tr><td rowspan="4">참여학생</td><td>학번</td><td>이름</td><td>전공</td></tr>
<tr><td>2021041017</td><td>김규현</td><td>소프트웨어학과</td></tr>
<tr><td>2023041082</td><td>Namsraijalbuu Bilguuntuguldur</td><td>소프트웨어학과</td></tr>
<tr><td></td><td></td><td></td></tr>
<tr><td>지도교수</td><td colspan="3">이종연</td></tr>
<tr><td colspan="4">작품(주제)에 대한 요약</td></tr>
<tr><td>작품 설명</td><td colspan="3">이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.</td></tr>
<tr><td>사용자 유형</td><td colspan="3">1. 주요 사용자 : 데이터베이스를 학습/개발하는 대학생 및 개발자
2. 보조 사용자 : 서버에 접속하는 외부 클라이언트(mysql CLI, DBeaver 등) 및 AI 어시스턴트(Claude Desktop)
3. 관리자 : 서버 운영 및 사용자/권한 관리 담당자</td></tr>
<tr><td>운영개념</td><td colspan="3">1. RuSQL 시스템은 Windows PC에서 관계형 데이터베이스 엔진 서버와 데스크톱 관리 도구로 나뉘어 운영된다.
2. 관리 도구와 외부 클라이언트는 TCP(자체 프로토콜, MySQL 프로토콜)로 서버에 접속하여 SQL을 수행하고, AI 어시스턴트는 MCP 서버를 통해 서버와 관리 도구를 조작한다.
3. 데이터는 로컬 디스크에 저장되며, 장애 발생 시 WAL(Write-Ahead Log)로 복구한다.</td></tr>
</table>

작품(주제)에 대한 요약 (계속)	
작품의 주요 기능	1. 시중의 여러 관계형 데이터베이스 시스템들이 지원하는 주요 범위의 SQL 쿼리 처리 및 실행을 지원한다. 2. B+Tree, Hash 인덱스와 비용 기반 쿼리 최적화로 질의를 빠르게 수행한다. 3. 행 단위 MVCC, 락, 데드락 감지 등으로 여러 사용자의 동시 접속을 처리하고, 트랜잭션 4단계 격리 수준을 지원한다. 4. WAL과 Undo Log로 트랜잭션의 원자성을 보장하고, 장애 시 데이터를 복구한다. 5. MySQL 프로토콜을 지원하여 기존 MySQL 클라이언트와 호환된다. 6. 소프트웨어의 관리 도구에서 SQL 편집·실행, 실행 계획 시각화, 스키마·ERD 조회와 편집, 서버 모니터링(세션, 락, 느린 쿼리 로그) 등을 제공한다. 7. MCP 도구들로 AI 어시스턴트가 자연어로 데이터베이스를 조작·진단하고 관리 도구의 화면을 조작한다.
기타 개발시 고려사항	1. ANSI/ISO SQL과 MySQL 프로토콜 문법을 기준으로 개발하여 기존 MySQL 클라이언트와 호환되도록 한다. 2. 모든 변경은 자동 Regression Test(946개 케이스)와 실제 MySQL 8.4.9와의 결과 대조, 실제 서버/클라이언트를 이용한 검증을 통과해야 한다. 3. 평문 비밀번호는 전송하지 않으며, DROP 같은 위험한 SQL은 AI가 사용자 확인 없이 실행하지 못하도록 보안 장치를 마련한다.
선행기술 조사 분석	가. 오픈소스·상용 DBMS 1. MySQL(InnoDB) : 행 단위 MVCC, Gap Lock, 공개된 클라이언트/서버 프로토콜을 참고하여 호환 기준으로 삼았다. 2. PostgreSQL : MVCC, SSI(직렬화 스냅샷 격리), 통계 기반 옵티마이저를 참고하였다. 3. SQLite : 단일 파일 기반 임베디드 DBMS로, 서버형 다중 세션과 쓰기 동시성이 제한적이다. 나. 논문 1. C. Mohan et al., "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging", ACM TODS, 1992 : WAL 기반 복구 2. M. J. Cahill et al., "Serializable Isolation for Snapshot Databases", SIGMOD, 2008 : SSI 3. P. G. Selinger et al., "Access Path Selection in a Relational Database

	Management System", SIGMOD, 1979 : 비용 기반 최적화 4. H. Berenson et al., "A Critique of ANSI SQL Isolation Levels", SIGMOD, 1995 : 격리 수준과 이상 현상(팬텀 등) 다. AI 연동 1. MCP(Model Context Protocol, Anthropic, 2024) : AI 어시스턴트가 외부 도구에 접속하는 개방형 표준이다. 일반적으로 공개된 MCP 데이터베이스 서버는 주로 질의/스키마 조회에 한정되나, 이 작품에서는 AI 에이전트가 행동을 할 수 있는 범위를 대폭 증강시켜 사용자와 친화적으로 구성하였고, 이에 대한 화면 조작과 위험 SQL 안전장치들까지 제공하여 보안을 강화하였다. 라. 차별점 : 기존 관계형 데이터베이스의 복잡한 시스템 및 핵심 기법을 직접 구현하여 검증하고, AI/MCP 기술을 통합하였다.
Key Words (5개) : 관계형 데이터베이스(RDBMS), MVCC, 비용 기반 쿼리 최적화, 트랜잭션, MCP	
지도 교수	(서명)

2026년 충북대학교 소프트웨어학부 졸업작품연구과제

[RuSQL]

프로젝트 관리 계획서 (요약)

2026년 11월 9일

문서번호 : 26-RuSQL-Doc-002
소 속 : 충북대학교 소프트웨어학과
팀 명 : RuSQL
팀 원 : 김규현, 빌궁
교 수 : 이종연 교수님

목 차

1. 프로젝트 개요 ---- 1
1.1 시스템 명칭 ---- 1
1.2 시스템 개요 ---- 1

2. 개발 프로세스 ---- 1

3. 팀 구성 : 역할 및 책임 ---- 2

4. 프로젝트 운영 관리 ---- 3
4.1 팀 미팅 계획 ---- 3
4.2 Ground Rules ---- 3

5. 개발 환경 ---- 3
5.1 하드웨어 환경 ---- 3
5.2 소프트웨어 환경 ---- 4
5.3 문서 작성도구 ---- 4

6. 산출물 목록 ---- 5

7. 기타 사항 ---- 5
8. 참고문헌 및 부록 ---- 5

1. 프로젝트 개요

1.1 시스템 명칭

1. 국문 : AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
2. 영문 : AI/MCP Integrated Relational Database Management System (RuSQL)
3. RuSQL은 관계형 데이터베이스 시스템에 AI/MCP 연동 기능을 통합한 시스템이다.

1.2 시스템 개요

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

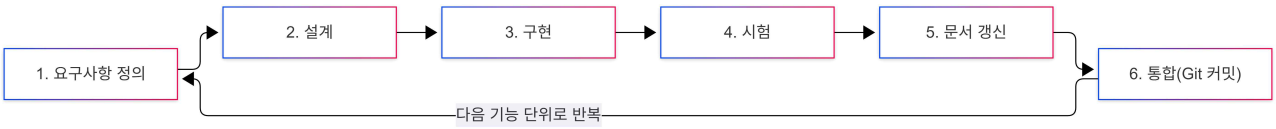
2. 개발 프로세스

2.1 프로세스 모형

1) 선택한 모형 : 점진적 모델(Incremental Model)
2) 선택 근거
- 요구사항 변경이 자주 발생한다.
- 시스템의 기능(SQL 처리, 인덱스, 동시성 제어, 관리 도구, MCP 연동)을 독립된 단위로 나누어 순차적으로 추가할 수 있다.
- 동시성 제어와 장애 복구처럼 기술적 위험이 큰 기능은 반복마다 자동 회귀 테스트와 실제 서버 검증으로 조기에 확인한다.

2.2 개발 절차

그림 1은 RuSQL의 점진적 개발 절차를 나타낸다.
그림 1. RuSQL 점진적 개발 절차



3. 팀 구성 : 역할 및 책임.

표 1은 팀원별 역할과 책임을 정의한다.

표 1. 팀 구성

이름	역할	책임	기타
김규현	팀장, 프로덕트 오너	1) 요구사항을 수집/정리하고 우선순위를 결정 2) 시스템을 설계하고 구현 3) 테스트를 수행하고 GitHub Repository, 문서를 관리	
Namsraijalbuu Bilguuntuguldur	팀원	1) 발표 및 AI 토큰 비용 관리 2) 산출물 문서 관리/검토	

4. 프로젝트 운영 관리

4.1 팀 미팅 계획

(1) 정기 회의
- 팀 미팅 시간 : 매주 화요일 16시
- 미팅시 토의 내용 : 개인별 수행 결과 보고, 문제점 제시 및 해결 방안 토의
- 미팅 참석자 : 지도교수, 팀원, 팀장

(2) 비정기 회의
- 프로젝트 진행에 급하게 회의가 필요한 경우에 진행한다.
- Zoom, Kakaotalk, Discord 등을 활용하여 비대면으로 진행한다.

(3) 프로젝트 발표 미팅
- 설계 프로젝트 교과목의 중간 발표와 최종 발표를 준비하고 진행한다.

4.2 Ground Rules

1. 개발 일정 : 팀원은 합의된 일정 안에 담당 작업을 완료한다. 일정을 지키지 못하는 팀원은 팀장에게 사전에 보고하고 일정을 재조정한다.
2. 회의 참석 : 팀원은 정기 회의에 참석한다. 불참하는 팀원은 회의 전에 팀장에게 사유를 알린다.
3. 코드 품질 : 팀원은 자동 회귀 테스트를 통과한 변경만 저장소에 반영한다.
4. 문서화 : 팀원은 기능을 변경한 반복 안에서 관련 문서를 함께 갱신한다.
5. 기여 : 팀은 새로운 아이디어를 제시하거나 기여도가 높은 팀원의 결과를 회의에서 공유하고 결과보고서에 반영한다.

5. 개발 환경

5.1 하드웨어 개발 환경

Windows PC

5.2 소프트웨어 개발 환경

표 2는 개발에 사용하는 소프트웨어의 명칭과 버전을 나타낸다.

표 2. 소프트웨어 개발 환경

구분	명칭	버전	용도
운영체제	Windows 11 Pro (64비트)	10.0.26300	개발 및 시험 환경
핵심 개발 언어	C++	C++20	데이터베이스 엔진, 서버, 터미널·클라이언트 도구
컴파일러	Visual Studio 2022 (MSVC)	17 (2022)	C++20 엔진 컴파일
빌드 도구	CMake	4.3.3	엔진 빌드
언어·도구	Rust (Cargo)	1.96.0	Tauri 백엔드
런타임	Node.js / npm	24.16.0 / 11.13.0	프론트엔드 빌드
프레임워크	Tauri	2.11.5 (CLI 2.11.2)	데스크톱 관리 도구
프레임워크	React / TypeScript / Vite	19.2.4 / 5.8.3 / 7.3.5	관리 도구 화면
라이브러리	Monaco Editor (@monaco-editor/react)	4.7.0	SQL 편집기
언어	Python	3.14.4	MCP 서버, 벤치마크 스크립트
오픈소스 라이브러리	nlohmann/json, LZ4, Catch2, SHA-1, PicoSHA2	3.11.3, 1.9.4, 2.13.10, -, -	JSON 처리, 압축, 단위 테스트, 인증 해시
비교·검증	MySQL Server / Client	8.4.9	호환성 및 성능 비교 검증
형상 관리	Git / GitHub	-	소스 코드 및 문서 관리
편집기	Visual Studio Code	-	코드 및 문서 편집
AI 도구	Claude Desktop	-	MCP 연동 시험

5.3 문서 작성 도구

1. 산출물 문서(D01 ~ D08) : 한글/PDF
2. 다이어그램 : draw.io, mermaid.ai

6. 산출물 목록

표 3은 산출물의 문서코드, 문서명, 작성 담당자, 완료 예정일을 정의한다.

표 3. 산출물 목록

순서	문서코드	문서명	작성 담당자	완료 예정일
1	RuSQL_D01_v1.0	시스템 정의서(프로젝트 요약서)	김규현, 빌궁	26.11.05
2	RuSQL_D02_v1.0	프로젝트 관리 계획서(요약)	김규현, 빌궁	26.11.05
3	RuSQL_D03_v1.0	소프트웨어 기능 아키텍처	김규현, 빌궁	26.09.28
4	RuSQL_D04_v1.0	프로덕트 백로그(요구사항 정의)	김규현, 빌궁	26.11.05
5	RuSQL_D05_v1.0	N차 스프린트 백로그(태스크 목록)	김규현, 빌궁	26.11.05
6	RuSQL_D06_v1.0	N차 스프린트 명세서	김규현, 빌궁	26.11.05
7	RuSQL_D07_v1.0	소프트웨어 시스템 시험 결과서	김규현, 빌궁	26.11.05
8	RuSQL_D08_v1.0	프로젝트 결과보고서	김규현, 빌궁	26.11.05

7. 기타 사항

형상 관리 : 팀은 소스 코드와 문서를 Git으로 관리하고 GitHub 저장소(https://github.com/rspstat/RuSQL)에 반영한다.

이전 구현 : 초기에 Rust로 구현하였던 버전은 저장소의 legacy 브랜치에 보존하며, 현재 C++로 포팅된 최신 버전은 main 브랜치에서 진행한다.

8. 참고문헌 및 부록

해당 없음.

<끝>

2026년 충북대학교 소프트웨어학부 졸업작품연구과제

[RuSQL]

소프트웨어 기능 아키텍처

2026년 9월 26일

문서번호 : 26-RuSQL-Doc-003
소 속 : 충북대학교 소프트웨어학과
팀 명 : RuSQL
팀 원 : 김규현, 빌궁
교 수 : 이종연 교수님

제/개정 이력

버전	날짜	작성자 성명	제/개정사항	비 고
0.1	26.09.28	김규현	첫 작성	

1. 프로젝트 개요

1.1 시스템명

1. 국문 : AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
2. 영문 : AI/MCP Integrated Relational Database Management System (RuSQL)

1.2 시스템 개요

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

2. 기능 아키텍처

2.1 액터 목록

표 1은 RuSQL의 액터와 역할을 정의한다.

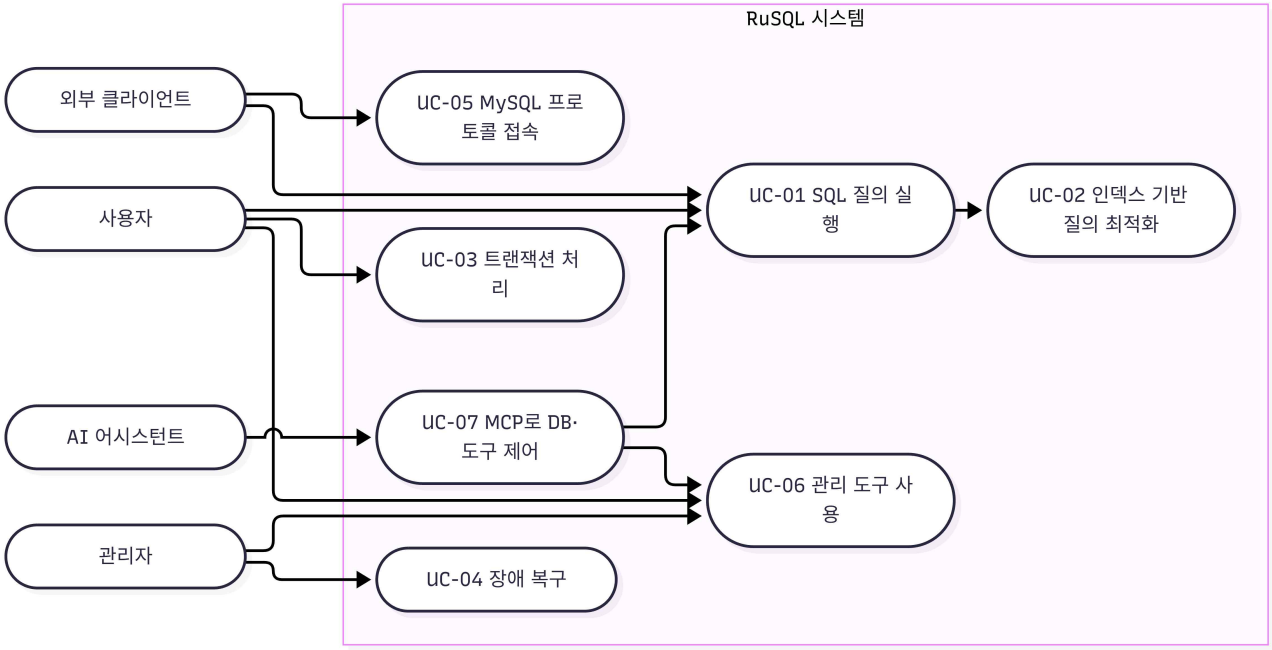
표 1. 액터 목록

액터명	역할	참고
사용자(User)	SQL을 실행하고 데스크톱 관리 도구로 데이터베이스를 조작하는 학생·개발자	
외부 클라이언트(External Client)	MySQL 프로토콜로 서버에 접속하는 mysql CLI, DBeaver 등	
AI 어시스턴트(AI Assistant)	MCP 도구로 서버와 관리 도구를 자연어로 조작하는 Claude Desktop 등	
관리자(Administrator)	서버를 기동·정지하고 장애 발생 시 복구를 확인하는 운영 담당자	본 프로젝트에서는 사용자가 겸임

2.2 사용사례 다이어그램

그림 1은 RuSQL의 액터별 주요 사용사례를 나타낸다.

그림 1. RuSQL 사용사례 다이어그램



2.3 사용사례 목록

표 2는 그림 1의 각 사용사례를 설명한다.

표 2. 사용사례 목록

UC ID	사용사례 명	관련 액터	설명
UC-01	SQL 질의 실행	사용자, 외부 클라이언트	SQL 문을 파싱하고 실행 계획을 세워 실행한 뒤 결과를 반환한다.
UC-02	인덱스 기반 질의 최적화	사용자	B+Tree·Hash 인덱스와 비용 기반 옵티마이저로 UC-01의 실행 계획을 선택한다.
UC-03	트랜잭션 처리	사용자	BEGIN/COMMIT/ROLLBACK과 행 단위 MVCC, 4단계 격리 수준으로 동시 접속을 처리한다.

UC-04	장애 복구	관리자	서버 재기동 시 WAL·Undo Log로 커밋된 변경을 재현하고 미완료 변경을 되돌린다.
UC-05	MySQL 프로토콜 접속	외부 클라이언트	기존 MySQL 클라이언트가 TCP로 접속하여 SQL을 수행한다.
UC-06	관리 도구 사용	사용자, 관리자	데스크톱 관리 도구에서 SQL 편집·실행, 스키마·ERD 조회, 서버 상태 모니터링을 수행한다.
UC-07	MCP로 DB·도구 제어	AI 어시스턴트	MCP 도구로 자연어 요청을 SQL 실행 또는 관리 도구 화면 조작으로 변환하여 수행한다.

3. 기타 요구사항

해당 없음.

<끝>

2026년 충북대학교 소프트웨어학부 졸업작품연구과제

[RuSQL]

프로덕트 백로그(Product Backlog)

(요구사항 정의)

2026년 11월 9일

문서번호 : 26-RuSQL-Doc-004
소 속 : 충북대학교 소프트웨어학과
팀 명 : RuSQL
팀 원 : 김규현, 빌궁
교 수 : 이종연 교수님

제/개정 이력

버전	날짜	작성자 성명	제/개정사항	비 고
0.1	26.09.28	김규현	첫 작성	
0.2	26.11.05	김규현	표 4의 머리글(추정작업량, 우선순위) 보완	

1. 프로젝트 개요

1.1 시스템명

1. 국문 : AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
2. 영문 : AI/MCP Integrated Relational Database Management System (RuSQL)

1.2 시스템 개요

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

2. 요구사항

2.1 주제+에픽 목록

표 3은 D03의 사용사례를 기반으로 식별한 주제와 에픽을 나타낸다.

표 3. 주제·에픽 목록

액터	주제ID	주제명	에픽ID	에픽명	에픽설명
사용자	T-01	SQL 처리	E-01	SQL 파싱·실행	사용자가 입력한 SQL을 파싱하고 실행 계획을 세워 실행한다.
			E-02	비용 기반 질의 최적화	인덱스와 통계 정보를 이용해 비용이 가장 낮은 실행 계획을 선택한다.
사용자	T-02	트랜잭션·동시성	E-03	MVCC 트랜잭션 처리	행 단위 MVCC와 격리 수준으로 동시 트랜잭션을 처리한다.
			E-04	락·데드락 감지	락 매니저로 자원 접근을 통제하고 데드락을 감지·해소한다.
관리자	T-03	장애 복구	E-05	WAL·Undo 복구	서버 재기동 시 WAL과 Undo Log로 데이터 정합성을 복구한다.

외부 클라이언트	T-04	MySQL 프로토콜 호환	E-06	MySQL 와이어 프로토콜 서버	기존 MySQL 클라이언트가 별도 드라이버 없이 접속하도록 지원한다.
사용자	T-05	관리 도구	E-07	SQL 편집기	데스크톱 도구에서 SQL을 작성·실행하고 결과를 확인한다.
			E-08	스키마·ERD 뷰어	테이블·컬럼·관계를 조회하고 ERD로 시각화한다.
			E-09	서버 모니터링	서버 상태와 연결 정보를 조회한다.
AI 어시스턴트	T-06	MCP·AI 연동	E-10	MCP로 DB 제어	AI가 MCP 도구로 SQL을 실행하고 결과를 조회한다.
			E-11	MCP로 관리 도구 제어	AI가 MCP 도구로 관리 도구의 화면(에디터 탭 등)을 조작한다.
			E-12	위험 SQL 안전장치	AI가 DROP 등 위험 SQL을 사용자 확인 없이 실행하지 못하게 한다.

2.2 사용자 스토리

표 4는 2.1절 에픽별 사용자 스토리를 정의한다.

표 4. 사용자 스토리

에픽ID	에픽명	US_ID	사용자 스토리	추정작업량	우선순위
E-01	SQL 파싱·실행	US-01	사용자는 다양한 범위의 SQL 쿼리문을 실행하기 위하여 SQL 쿼리문을 서버에 전달한다.	8	상
		US-02	사용자는 여러 테이블을 조합한 결과를 얻기 위하여 JOIN·서브쿼리·CTE가 포함된 SQL 문을 실행한다.	8	상

E-02	비용 기반 질의 최적화	US-03	사용자는 대량 데이터에서 빠른 응답을 얻기 위하여 B+Tree·Hash 인덱스를 생성한다.	5	상
		US-04	사용자는 실행 계획을 확인하기 위하여 EXPLAIN으로 질의 계획을 조회한다.	3	중
E-03	MVCC 트랜잭션 처리	US-05	사용자는 여러 세션에서 안전하게 데이터를 갱신하기 위하여 BEGIN/COMMIT/ROLLBACK으로 트랜잭션을 실행한다.	8	상
		US-06	사용자는 트랜잭션의 데이터 정합성 수준을 선택하기 위하여 4단계 격리 수준 중 하나를 지정한다.	5	중
E-04	락·데드락 감지	US-07	사용자는 동시 갱신 충돌 시 시스템이 정지하지 않도록 데드락이 자동으로 감지·해소되기를 기대한다.	5	중
E-05	WAL·Undo 복구	US-08	관리자는 서버 비정상 종료 후에도 데이터를 보존하기 위하여 재기동 시 자동 복구가 수행되기를 기대한다.	8	상
E-06	MySQL 와이어 프로토콜 서버	US-09	외부 클라이언트 사용자는 기존 도구를 그대로 쓰기 위하여 mysql CLI·DBeaver로 서버에 접속한다.	5	상
E-07	SQL 편집기	US-10	사용자는 SQL을 편리하게 작성하기 위하여 관리 도구의 편집기에서 SQL을 작성·실행한다.	5	상
E-08	스키마·ERD 뷰어	US-11	사용자는 테이블 구조를 파악하기 위하여 스키마와 ERD를 조회한다.	3	중
E-09	서버 모니터링	US-12	관리자는 서버 상태를 확인하기 위하여 연결 목록과 서버 상태를 조회한다.	2	중
E-10	MCP로 DB 제어	US-13	AI 어시스턴트는 사용자를 대신해 SQL을 실행하기 위하여 MCP execute_sql	5	상

			도구를 호출한다.		
E-11	MCP로 관리 도구 제어	US-14	AI 어시스턴트는 사용자의 작업을 돕기 위하여 MCP 도구로 편집기 탭을 생성·전환하고 SQL을 기입한다.	5	중
E-12	위험 SQL 안전 장치	US-15	사용자는 실수로 데이터를 잃지 않기 위하여 AI가 DROP 등 위험 SQL 실행 전에 확인을 요청하기를 기대한다.	3	상

3. 기타 사항

해당 없음.

<끝>

2026년 충북대학교 소프트웨어학부 졸업작품연구과제

[RuSQL]

[1]차 스프린트 백로그
(개발 태스크 목록)

2026년 11월 9일

문서번호 : 26-RuSQL-Doc-005
소 속 : 충북대학교 소프트웨어학과
팀 명 : RuSQL
팀 원 : 김규현, 빌궁
교 수 : 이종연 교수님

1. 프로젝트 개요

1.1 시스템명

1. 국문 : AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
2. 영문 : AI/MCP Integrated Relational Database Management System (RuSQL)

1.2 시스템 개요

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

2. [1]차 스프린트 - 요구사항

3.1 [1]차 스프린트 개발기간 : From (2026-03-02) To (2026-12-31)

3.2 요구사항 목록

본 프로젝트는 하나의 스프린트로 전체 개발을 진행하므로, 1차 스프린트의 범위는 D04 프로덕트 백로그의 사용자 스토리 전체(US-01 ~ US-15)와 같다. 표 1은 D04의 사용자 스토리를 에픽별로 인용한 것이다.

표 1. 1차 스프린트 요구사항 목록

에픽ID	에픽명	US_ID	사용자 스토리	추정작업량	우선순위	참고
E-01	SQL 파싱·실행	US-01	사용자는 다양한 범위의 SQL 쿼리문을 실행하기 위하여 SQL 쿼리문을 서버에 전달한다.	8	상	
		US-02	사용자는 여러 테이블을 조합한 결과를 얻기 위하여 JOIN·서브쿼리·CTE가 포함된 SQL 문을 실행한다.	8	상	
E-02	비용 기반 질의 최적화	US-03	사용자는 대량 데이터에서 빠른 응답을 얻기 위하여 B+Tree·Hash 인덱스를 생성한다.	5	상	
		US-04	사용자는 실행 계획을 확인하기 위하여 EXPLAIN으로 질의 계획을 조회한다.	3	중	

에픽ID	에픽명	US_ID	사용자 스토리	추정작업량	우선순위	참고
E-03	MVCC 트랜잭션 처리	US-05	사용자는 여러 세션에서 안전하게 데이터를 갱신하기 위하여 BEGIN/COMMIT/ROLLBACK으로 트랜잭션을 실행한다.	8	상	
		US-06	사용자는 트랜잭션의 데이터 정합성 수준을 선택하기 위하여 4단계 격리 수준 중 하나를 지정한다.	5	중	
E-04	락·데드락 감지	US-07	사용자는 동시 갱신 충돌 시 시스템이 정지하지 않도록 데드락이 자동으로 감지·해소되기를 기대한다.	5	중	
E-05	WAL·Undo 복구	US-08	관리자는 서버 비정상 종료 후에도 데이터를 보존하기 위하여 재기동 시 자동 복구가 수행되기를 기대한다.	8	상	
E-06	MySQL 와이어 프로토콜 서버	US-09	외부 클라이언트 사용자는 기존 도구를 그대로 쓰기 위하여 mysql CLI·DBeaver로 서버에 접속한다.	5	상	
E-07	SQL 편집기	US-10	사용자는 SQL을 편리하게 작성하기 위하여 관리 도구의 편집기에서 SQL을 작성·실행한다.	5	상	
E-08	스키마·ERD 뷰어	US-11	사용자는 테이블 구조를 파악하기 위하여 스키마와 ERD를 조회한다.	3	중	
E-09	서버 모니터링	US-12	관리자는 서버 상태를 확인하기 위하여 연결 목록과 서버 상태를 조회한다.	2	중	
E-10	MCP로 DB 제어	US-13	AI 어시스턴트는 사용자를 대신해 SQL을 실행하기 위하여 MCP execute_sql 도구를 호출한다.	5	상	
E-11	MCP로 관리 도구 제어	US-14	AI 어시스턴트는 사용자의 작업을 돕기 위하여 MCP 도구로 편집기 탭을 생성·전환하고 SQL을 기입한다.	5	중	
E-12	위험 SQL 안전장치	US-15	사용자는 실수로 데이터를 잃지 않기 위하여 AI가 DROP 등 위험 SQL 실행 전에 확인을 요청하기를 기대한다.	3	상	

3. [1]차 스프린트 - 태스크

3.1 태스크 목록

표 2는 2절의 요구사항을 구현하기 위한 태스크 목록이다.

표 2. 태스크 목록

US_ID	스토리(요약)	TK_ID	테스크 명	담당자	종료일	비고
US-01	SQL 실행	TK-01	렉서·파서·AST 구현	김규현	07/08	C++ 재작성
		TK-02	실행기(DDL·DML·SELECT) 구현	김규현	07/08	
US-02	JOIN·서브쿼리·CTE	TK-03	조인 5종·서브쿼리·CTE·윈도우 구현	김규현	08/27	역방향 NL
US-03	인덱스 생성	TK-04	B+Tree·Hash·복합 인덱스 구현	김규현	08/05	증분 갱신
US-04	실행 계획 조회	TK-05	비용 기반 플래너·EXPLAIN 구현	김규현	10/25	
US-05	트랜잭션	TK-06	행 단위 MVCC·트랜잭션 매니저 구현	김규현	08/04	
US-06	격리 수준	TK-07	4단계 격리 수준·Gap Lock·SSI 구현	김규현	09/28	팬텀 감지 보강
US-07	데드락 해소	TK-08	락 매니저·데드락 감지 구현	김규현	08/12	
US-08	장애 복구	TK-09	WAL·redo 로그·크래시 복구 구현	김규현	11/01	redo 범위 확대
US-09	MySQL 프로토콜	TK-10	MySQL 와이어 프로토콜 서버 구현	김규현	07/08	
		TK-11	MySQL 8.4.9 대조 검증(49종)	김규현	11/09	차이 0건
US-10	SQL 편집기	TK-12	SQL 편집기·결과 그리드 구현	김규현	10/31	
US-11	스키마·ERD	TK-13	스키마 트리·ERD 편집기 구현	김규현	10/21	
US-12	서버 모니터링	TK-14	서버 매니저(세션·락·로그) 구현	김규현	10/31	
US-13	MCP DB 제어	TK-15	MCP 서버 DB 도구 구현	김규현	11/07	37종
US-14	MCP 관리 도구 제어	TK-16	MCP 편집기·앱 제어 도구 구현	김규현	09/21	
US-15	위험 SQL 확인	TK-17	위험 SQL 확인 절차 구현	김규현	09/21	
전체	US-01 ~ US-15	TK-18	자동 회귀 테스트(Catch2) 작성 및 실행	김규현	11/09	946 케이스
		TK-19	성능 측정과 개선, 성능 작업 동결	김규현	10/03	
		TK-20	산출물 문서(D01~D08) 작성	김규현, 빌궁	11/09	
		TK-21	중간·최종 발표 준비	김규현, 빌궁		
		TK-22	AI 토큰 비용 관리	빌궁	상시	

4. 소요 기술

표 3은 개발에 필요한 구현 기술과 지원 도구의 준비 현황이다.

표 3. 소요 기술

번호	도구 / 환경	담당자	완료일	비고
1	C++ · CMake · Visual Studio 2022(MSVC) 빌드 환경	김규현	07/07	엔진·서버 빌드
2	Rust(Cargo) · Tauri · React · TypeScript · Vite 환경	김규현	03/16	데스크톱 관리 도구
3	Python 3.14 · MCP SDK(FastMCP) 환경	김규현	06/07	AI 연동(MCP 서버)
4	Catch2 자동 회귀 테스트 프레임워크	김규현	07/08	회귀 테스트
5	Git · GitHub 저장소(legacy · main 브랜치)	김규현	03/16	소스·문서 관리
6	MySQL Server · Client 8.4.9	김규현	10/12	호환성 대조 검증
7	DBMS 이론 학습(ARIES, SSI, 비용 기반 최적화, InnoDB 락)	김규현		선행기술 조사는 D01 참고

5. 기타 사항

1. 본 프로젝트는 스프린트를 나누지 않고 하나의 반복(점진적 모델, D02 2.1절)으로 진행하므로 1차 스프린트가 D04의 사용자 스토리 전체를 담는다.
2. XA 분산 트랜잭션, 페이지 단위 버퍼 풀 교체 알고리즘, WAL 기반 복제(replication)는 2026-08-09 범위 검토에 따라 개발 범위에서 제외한다.
3. AI 기능은 사설 NL→SQL 모델 대신 Claude Desktop과 MCP 서버(도구 37종)로 구현한다.
4. 이 문서는 작성 시점(2026-11-09)의 현황이며, 개발이 계속되는 동안 변경된 사항은 D06~D08에 반영한다.
5. 본 문서에 나타나지 않은 사항은 상위 문서인 프로젝트 관리 계획서(D02)에 따라 진행한다.

<끝>

2026년 충북대학교 소프트웨어학부 졸업작품연구과제

[RuSQL]

[1]차 스프린트 - 개발 명세서

2026년 11월 9일

문서번호 : 26-RuSQL-Doc-006

소 속 : 충북대학교 소프트웨어학과

팀 명 : RuSQL

팀 원 : 김규현, 빌궁

교 수 : 이종연 교수님

목 차

1. 프로젝트 개요 ---- 1

2. 태스크 목록 ---- 1

3. 기술 명세 ---- 3
   3.1 클래스 명세 ---- 3
   3.2 핵심 알고리즘 명세 ---- 11
   3.3 DB 스키마 명세 ---- 15
   3.4 API 명세 ---- 16

4. 사용자 인터페이스(GUI) ---- 20

5. 기타 참고 사항 ---- 23

1. 프로젝트 개요

1.1 시스템명

1. 국문 : AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
2. 영문 : AI/MCP Integrated Relational Database Management System (RuSQL)

1.2 시스템 개요

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

2. 태스크 목록

D05의 표 2 태스크 목록에 완료율을 더하여 표 1과 같이 나타낸다. 완료율은 2026-11-09 기준이다.

표 1. 태스크 목록 및 완료율

US_ID	스토리(요약)	TK_ID	테스크 명	담당자	종료일	완료%
US-01	SQL 실행	TK-01	렉서·파서·AST 구현	김규현	07/08	100
		TK-02	실행기(DDL·DML·SELECT) 구현	김규현	07/08	100
US-02	JOIN·서브쿼리·CTE	TK-03	조인 5종·서브쿼리·CTE·윈도우 구현	김규현	08/27	100
US-03	인덱스 생성	TK-04	B+Tree·Hash·복합 인덱스 구현	김규현	08/05	100
US-04	실행 계획 조회	TK-05	비용 기반 플래너·EXPLAIN 구현	김규현	10/25	100
US-05	트랜잭션	TK-06	행 단위 MVCC·트랜잭션 매니저 구현	김규현	08/04	100
US-06	격리 수준	TK-07	4단계 격리 수준·Gap Lock·SSI 구현	김규현	09/28	100
US-07	데드락 해소	TK-08	락 매니저·데드락 감지 구현	김규현	08/12	100
US-08	장애 복구	TK-09	WAL·redo 로그·크래시 복구 구현	김규현	11/01	100
US-09	MySQL 프로토콜	TK-10	MySQL 와이어 프로토콜 서버 구현	김규현	07/08	100
		TK-11	MySQL 8.4.9 대조 검증(49종)	김규현	11/09	100
US-10	SQL 편집기	TK-12	SQL 편집기·결과 그리드 구현	김규현	10/31	100
US-11	스키마·ERD	TK-13	스키마 트리·ERD 편집기 구현	김규현	10/21	100
US-12	서버 모니터링	TK-14	서버 매니저(세션·락·로그) 구현	김규현	10/31	100
US-13	MCP DB 제어	TK-15	MCP 서버 DB 도구 구현	김규현	11/07	100
US-14	MCP 관리 도구 제어	TK-16	MCP 편집기·앱 제어 도구 구현	김규현	09/21	100
US-15	위험 SQL 확인	TK-17	위험 SQL 확인 절차 구현	김규현	09/21	100

US_ID	스토리(요약)	TK_ID	테스크 명	담당자	종료일	완료%
전체	US-01 ~ US-15	TK-18	자동 회귀 테스트(Catch2) 작성 및 실행	김규현	11/09	100
		TK-19	성능 측정과 개선, 성능 작업 동결	김규현	10/03	100
		TK-20	산출물 문서(D01~D08) 작성	김규현, 빌궁	11/09	
		TK-21	중간·최종 발표 준비	김규현, 빌궁		
		TK-22	AI 토큰 비용 관리	빌궁	상시	

3. 기술 명세

3.1 클래스 명세

그림 1은 RuSQL 엔진 라이브러리와 서버의 클래스 다이어그램이다. 다이어그램의 클래스 21개를 C-001부터 C-021까지 [번호] 클래스 : 클래스ID 이름 순으로 명세한다. 표의 타입 열은 멤버를 소스 코드에서 공개(Public)했는지 비공개(Private)했는지를 나타낸다.

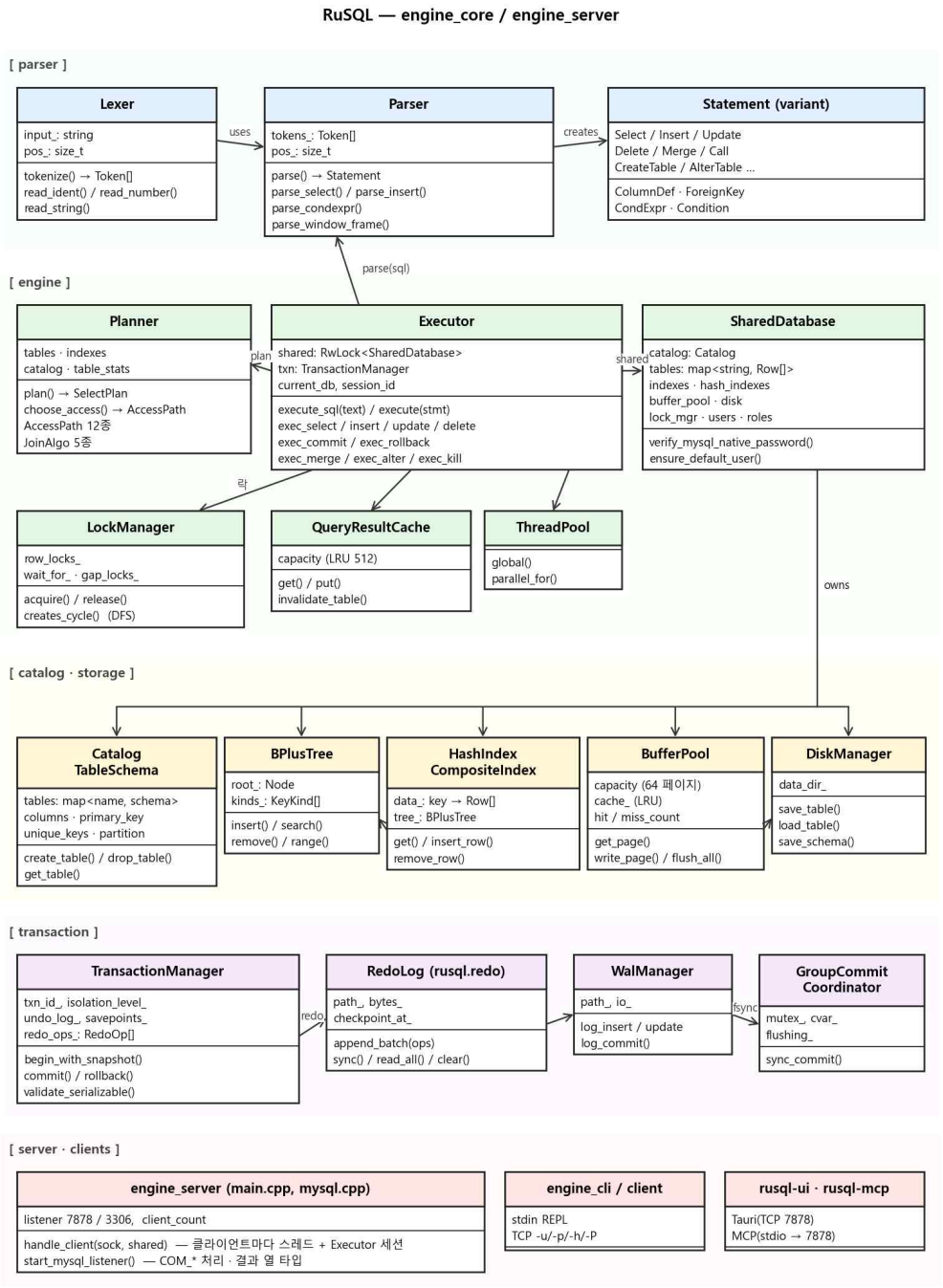


그림 1. RuSQL 클래스 다이어그램 (엔진 라이브러리와 서버)

[1] 클래스 : C-001 Lexer

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-001	멤버 변수	input_	SQL 원문 문자열	Private	
		pos_	현재 읽고 있는 위치	Private	
	멤버 함수	tokenize()	SQL 문자열을 토큰 목록(vector<Token>)으로 나눈다.	Public	
		read_ident() / read_number() / read_string()	식별자, 숫자, 문자열 리터럴 토큰을 읽는다.	Private	
		skip_whitespace()	공백과 주석을 건너뛴다.	Private	

[2] 클래스 : C-002 Parser

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-002	멤버 변수	tokens_	Lexer가 만든 토큰 목록	Private	
		pos_	현재 분석 중인 토큰 위치	Private	
		window_defs_	이름이 붙은 WINDOW 정의	Private	
	멤버 함수	parse()	토큰을 구문 분석하여 Statement(AST)를 만든다. 실패하면 ParseError를 반환한다.	Public	
		parse_select() / parse_insert() / parse_update() / parse_delete()	문장 종류별로 구문을 분석한다.	Private	
		parse_condexpr()	WHERE 조건식을 NOT, AND, OR 우선순위로 분석한다.	Private	
		parse_window_frame()	윈도우 함수의 프레임 절을 분석한다.	Private	

[3] 클래스 : C-003 Statement

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-003	멤버 변수	Statement (variant)	Select, Insert, Update, Delete, CreateTable, AlterTable, Begin, Commit, Rollback, Merge, Call 등 문장 종류를 std::variant로 보관한다.	Public	
		ColumnDef	열 이름, 데이터 타입, PK, NOT NULL, UNIQUE, AUTO_INCREMENT, 기본값, CHECK 식	Public	
		ForeignKey	외래키 열, 참조 표와 열, ON DELETE / ON UPDATE 동작(Restrict, Cascade, SetNull, SetDefault)	Public	
		CondExpr	조건식 트리(And, Or, Not, Leaf)	Public	
	멤버 함수	(멤버 함수 없음)	AST는 데이터만 담는 구조체이다.	-	

[4] 클래스 : C-004 Planner

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-004	멤버 변수	tables, indexes, catalog, table_stats	계획 수립에 읽는 표, 인덱스, 카탈로그, 통계 정보	Public	
		SelectPlan	기본 표 계획(base)과 조인 계획 목록(joins)	Public	
	멤버 함수	plan(table, condition, joins)	접근 경로, 조인 알고리즘, 조인 순서를 정해 SelectPlan을 만든다.	Public	
		choose_access(table, cond, pk)	후보 AccessPath(전체 스캔, PK·보조·해시·복합 인덱스 등 12종) 가운데 비용이 가장 낮은 것을 고른다.	Public	
		estimate_rows() / estimate_cost()	통계로 행 수와 비용을 추정한다. 조인은 5종(중첩 루프, 해시, 정렬 병합, 인덱스 NL, 역방향 인덱스 NL) 중에서 고른다.	Public	

[5] 클래스 : C-005 Executor

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-005	멤버 변수	shared	세션들이 함께 쓰는 SharedDatabase (shared_ptr<RwLock<SharedDatabase>>)	Public	
		txn	이 세션의 TransactionManager	Public	
		current_db, session_id, auth_user	현재 데이터베이스, 세션 번호, 로그인한 사용자	Public	
	멤버 함수	execute_sql(text)	Lexer, Parser, execute를 차례로 호출하여 SQL 한 문장을 실행한다.	Public	
		execute(Statement)	문장 종류에 맞는 exec_* 함수를 호출하고 결과 문자열을 만든다.	Public	
		exec_select() / exec_insert() / exec_update() / exec_delete() / exec_merge()	DML 문장을 실행한다.	Private	
		exec_begin() / exec_commit() / exec_rollback() / exec_savepoint()	트랜잭션을 제어한다.	Private	
		exec_suggest_indexes() / exec_kill()	인덱스 제안, 실행 중인 문장 중단을 처리한다.	Public	
		recover_from_redo()	서버 기동 시 redo 로그를 읽어 커밋된 변경을 다시 반영한다.	Private	

[6] 클래스 : C-006 SharedDatabase

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-006	멤버 변수	catalog, tables	표 정의와 표별 행(map<string, vector<Row>>)	Public	
		indexes, hash_indexes, composite_indexes	보조·해시·복합 인덱스	Public	
		buffer_pool, disk, lock_mgr	페이지 캐시, 디스크 관리자, 락 매니저	Public	
		users, grants, roles	계정, 권한, 역할	Public	
	멤버 함수	verify_mysql_native_password()	MySQL 접속 때 받은 비밀번호 응답을 검증한다.	Public	
		ensure_default_user()	사용자가 없으면 기본 계정을 만든다.	Public	

[7] 클래스 : C-007 LockManager

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-007	멤버 변수	row_locks_	(표, 기본키)별로 잠금을 쥔 트랜잭션	Private	
		wait_for_	트랜잭션이 기다리는 대상(대기 그래프)	Private	
		gap_locks_, predicate_reads_	Gap Lock과 SERIALIZABLE 검증용 읽기 조건	Private	
	멤버 함수	acquire(table, pk, txn_id) / acquire_shared()	행 배타·공유 락을 건다. 충돌하면 대기한다.	Public	
		acquire_gap()	범위를 잠가 다른 트랜잭션의 INSERT를 막는다.	Public	
		creates_cycle(from, to)	대기 그래프를 깊이 우선으로 따라가 데드락 여부를 판정한다.	Private	
		release(txn_id)	트랜잭션이 쥔 락을 모두 풀고 기다리던 세션을 깨운다.	Public	

[8] 클래스 : C-008 QueryResultCache

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-008	멤버 변수	capacity	캐시 용량(LRU 512개)	Public	
		entries_, table_to_keys_	저장한 결과와 표별 캐시 키	Private	
	멤버 함수	get() / put()	같은 문장의 결과를 꺼내거나 저장한다.	Public	
		invalidate_table()	표가 바뀌면 그 표의 결과를 버린다.	Public	

[9] 클래스 : C-009 ThreadPool

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-009	멤버 변수	(프로세스 전역 풀)	서버 전체가 하나의 스레드 풀을 공유한다.	Private	
	멤버 함수	global()	전역 스레드 풀을 돌려준다.	Public	
		parallel_for(n, fn)	WHERE 필터, GROUP BY, 정렬, 집계를 여러 스레드로 나누어 실행한다.	Public	

[10] 클래스 : C-010 Catalog / TableSchema

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-010	멤버 변수	tables	표 이름에서 TableSchema로의 맵	Public	
		columns, primary_key_columns	열 정의 목록과 기본키 열	Public	
		check_constraints, unique_keys	CHECK 제약과 UNIQUE 키	Public	
		auto_increment_counters, partition_info	AUTO_INCREMENT 카운터와 파티션 정보	Public	
	멤버 함수	create_table() / drop_table()	표 메타데이터를 만들고 지운다.	Public	
		get_table() / get_table_mut()	표 메타데이터를 읽거나 고치기 위해 가져온다.	Public	

[11] 클래스 : C-011 BPlusTree

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-011	멤버 변수	root_	루트 노드(InternalNode 또는 LeafNode)	Private	
		kinds_	키 종류(숫자 키, 글자 키) 구분	Private	
	멤버 함수	insert(key, value) / remove(key)	키를 넣고 뺀다. 노드가 넘치면 쪼개고 비면 균형을 다시 맞춘다.	Public	
		search(key)	키 하나를 찾는다.	Public	
		range_search() / scan_from() / scan_to()	키 범위를 순서대로 읽는다.	Public	

[12] 클래스 : C-012 HashIndex

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-012	멤버 변수	table, column	인덱스가 걸린 표와 열	Public	
		data_	키에서 행 목록으로의 맵	Private	
	멤버 함수	get(key)	같은 키를 가진 행을 바로 찾는다.	Public	
		insert_row() / remove_row()	행을 넣거나 뺄 때 인덱스를 함께 고친다.	Public	
		rebuild()	표의 행으로 인덱스를 다시 만든다.	Public	

[13] 클래스 : C-013 CompositeIndex

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-013	멤버 변수	table, columns, pk_columns	인덱스가 걸린 표, 열 목록, 기본키 열	Public	
		tree_	여러 열을 이어 만든 키를 가진 BPlusTree	Private	
	멤버 함수	search_exact()	모든 열이 같은 행을 찾는다.	Public	
		rows_with_prefix()	앞쪽 열만 같은 행을 찾는다.	Public	
		insert_row() / remove_row() / rebuild()	행 변경에 맞춰 인덱스를 고치거나 다시 만든다.	Public	

[14] 클래스 : C-014 BufferPool

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-014	멤버 변수	capacity	캐시할 표 수(기본 64)	Public	
		cache_	표별 행 묶음(LRU 순서)	Private	
		hit_count, miss_count	캐시 적중과 실패 횟수	Public	
	멤버 함수	get_page(table, disk)	캐시에서 표의 행을 읽고, 없으면 DiskManager에서 불러온다.	Public	
		write_page() / flush_all()	변경된 표를 디스크에 쓴다.	Public	
		hit_rate()	캐시 적중률을 계산한다.	Public	

[15] 클래스 : C-015 DiskManager

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-015	멤버 변수	data_dir_	데이터 디렉터리 경로	Private	
	멤버 함수	save_table() / load_table()	표 파일(.rdb, LZ4 압축)을 쓰고 읽는다.	Public	
		save_schema() / load_schema()	표 스키마(JSON)를 쓰고 읽는다.	Public	
		list_databases()	데이터 디렉터리의 데이터베이스 목록을 돌려준다.	Public	

[16] 클래스 : C-016 TransactionManager

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-016	멤버 변수	txn_id_, active_	현재 트랜잭션 번호와 진행 여부	Private	
		isolation_level_, frozen_ctx_	격리 수준과 그 수준에 맞는 스냅샷	Private	
		undo_log_, savepoints_	되돌리기용 기록과 세이브포인트	Private	
		read_set_, redo_ops_	SERIALIZABLE 검증용 읽은 행, 커밋 때 쓸 변경 기록	Private	
	멤버 함수	begin_with_snapshot()	트랜잭션 번호를 주고 스냅샷을 만든다. 스냅샷은 BEGIN이 아니라 첫 읽기에서 뜬다.	Public	
		commit() / rollback()	변경을 확정하거나 undo 기록으로 되돌린다.	Public	
		validate_serializable()	SERIALIZABLE에서 읽은 행이 바뀌었는지 검사한다.	Public	

[17] 클래스 : C-017 RedoLog

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-017	멤버 변수	path_	redo 로그 파일(rusql.redo) 경로	Private	
		bytes_, checkpoint_at_	파일 크기와 마지막 체크포인트 위치	Private	
	멤버 함수	append_batch(ops)	커밋한 트랜잭션의 변경(RedoOp: InsertVersion, SetXmax, TableFlushed)을 체크섬과 함께 덧붙인다.	Public	
		sync()	로그를 디스크에 fsync한다.	Public	
		read_all() / clear()	복구 때 전체를 읽고, 체크포인트 뒤에 비운다.	Public	

[18] 클래스 : C-018 WalManager

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-018	멤버 변수	path_	WAL 파일 경로	Private	
	멤버 함수	log_insert() / log_update() / log_delete()	변경 내용을 로그에 기록한다.	Public	
		log_commit() / log_rollback() / log_checkpoint()	트랜잭션 결과와 체크포인트를 기록한다.	Public	
		needs_auto_checkpoint()	로그가 커지면 자동 체크포인트가 필요한지 알려준다.	Public	

[19] 클래스 : C-019 GroupCommitCoordinator

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-019	멤버 변수	mutex_, cvar_	리더와 팔로워를 맞추는 뮤텍스와 조건 변수	Private	
		flushing_, generation_	fsync 진행 여부와 묶음 번호	Private	
	멤버 함수	sync_commit()	동시에 커밋하는 세션 중 하나(리더)가 fsync 한 번을 하고 나머지(팔로워)는 그 완료를 기다린다.	Public	

[20] 클래스 : C-020 engine_server

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-020	멤버 변수	listener	자체 프로토콜(7878)과 MySQL 프로토콜(3306) 수신 소켓	Private	
		client_count, shared	접속 수와 모든 세션이 공유하는 SharedDatabase	Private	
	멤버 함수	handle_client(sock, shared)	접속마다 스레드를 만들고 Executor 세션으로 SQL을 실행한다.	Public	
		start_mysql_listener()	MySQL 핸드셰이크, 인증, COM_* 명령을 처리한다.	Public	
		wire_error()	오류 메시지에서 MySQL 오류 번호와 SQLSTATE를 정한다.	Private	

[21] 클래스 : C-021 KillScope

C_ID	분류	멤버 변수/함수	설명	타입	참고
C-021	멤버 변수	was_flag, was_read_flag	문장 안에서 문장을 실행할 때 되돌릴 바깥 문장의 중단 요청 상태이다.	Public	
	멤버 함수	KillScope() / ~KillScope()	만들어질 때 이 스레드의 중단 요청 상태를 걸고, 사라질 때 되돌린다. tick() / check_now()는 같은 헤더(engine::deadline)의 일반 함수이다.	Public	

3.2 핵심 알고리즘 명세

앞서 3.1절의 클래스 가운데 처리 로직이 핵심인 메소드 4개의 행위를 그림 2부터 그림 5의 시퀀스 다이어그램과 단계로 명세한다.

[1] 메소드 : execute_sql( ) @ C-005 Executor

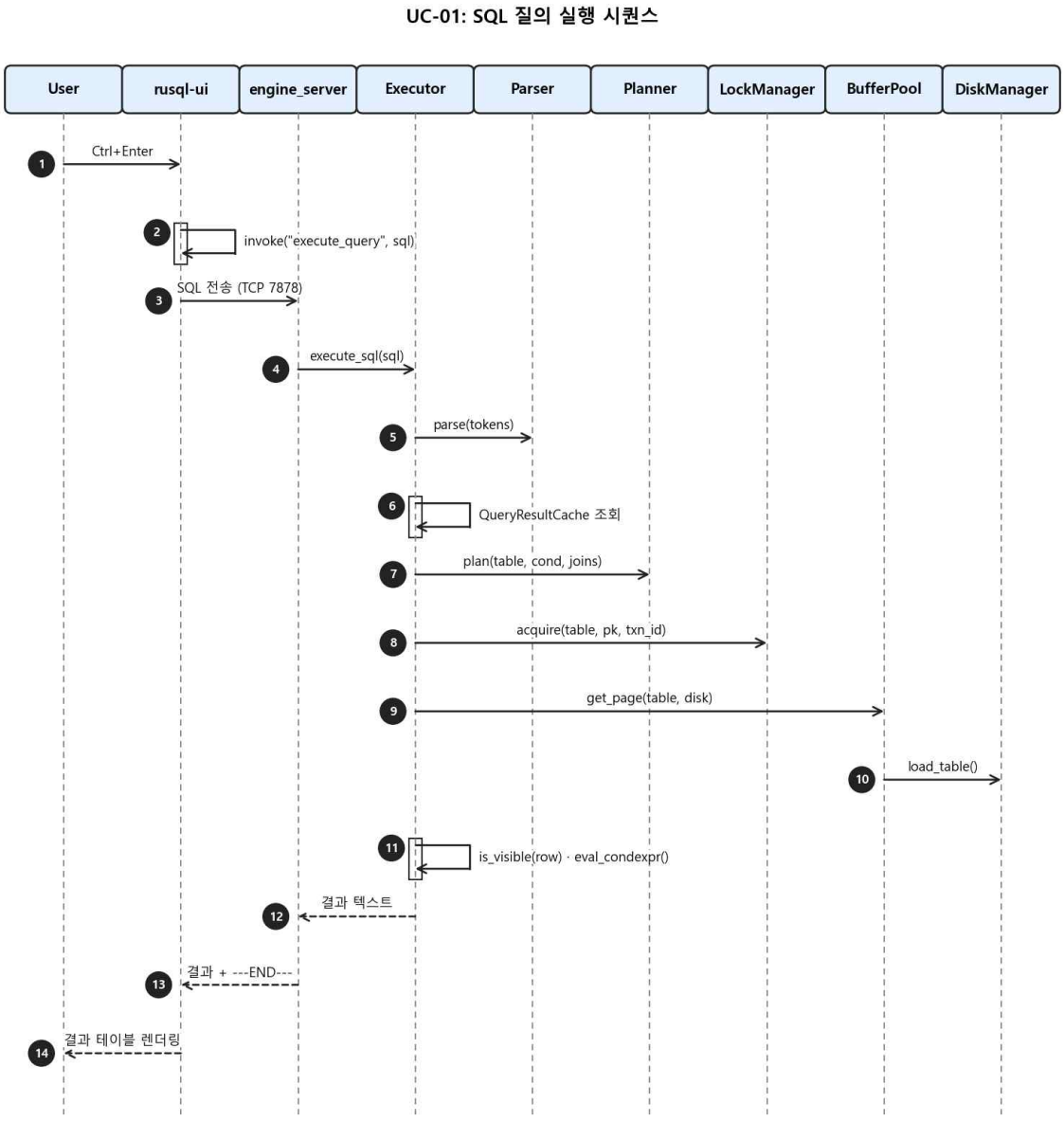


그림 2. execute_sql( ) 시퀀스

1. engine_server가 받은 SQL 문자열을 세션의 Executor에 넘긴다.
2. Lexer가 토큰으로 나누고 Parser가 Statement(AST)를 만든다.
3. 같은 문장과 같은 표 상태의 결과가 QueryResultCache에 있으면 그대로 돌려준다.
4. Planner가 접근 경로, 조인 알고리즘, 조인 순서를 정한다.
5. 쓰기 문장은 LockManager로 행 락을 얻는다. 충돌하면 대기하고 데드락을 검사한다.
6. BufferPool에서 행을 읽고(없으면 DiskManager가 .rdb를 읽는다) MVCC 가시성과 WHERE 조건을 확인한다.
7. 결과 문자열을 engine_server에 돌려주고, engine_server가 "---END---"와 함께 클라이언트에 보낸다.

[2] 메소드 : choose_access( ) @ C-004 Planner

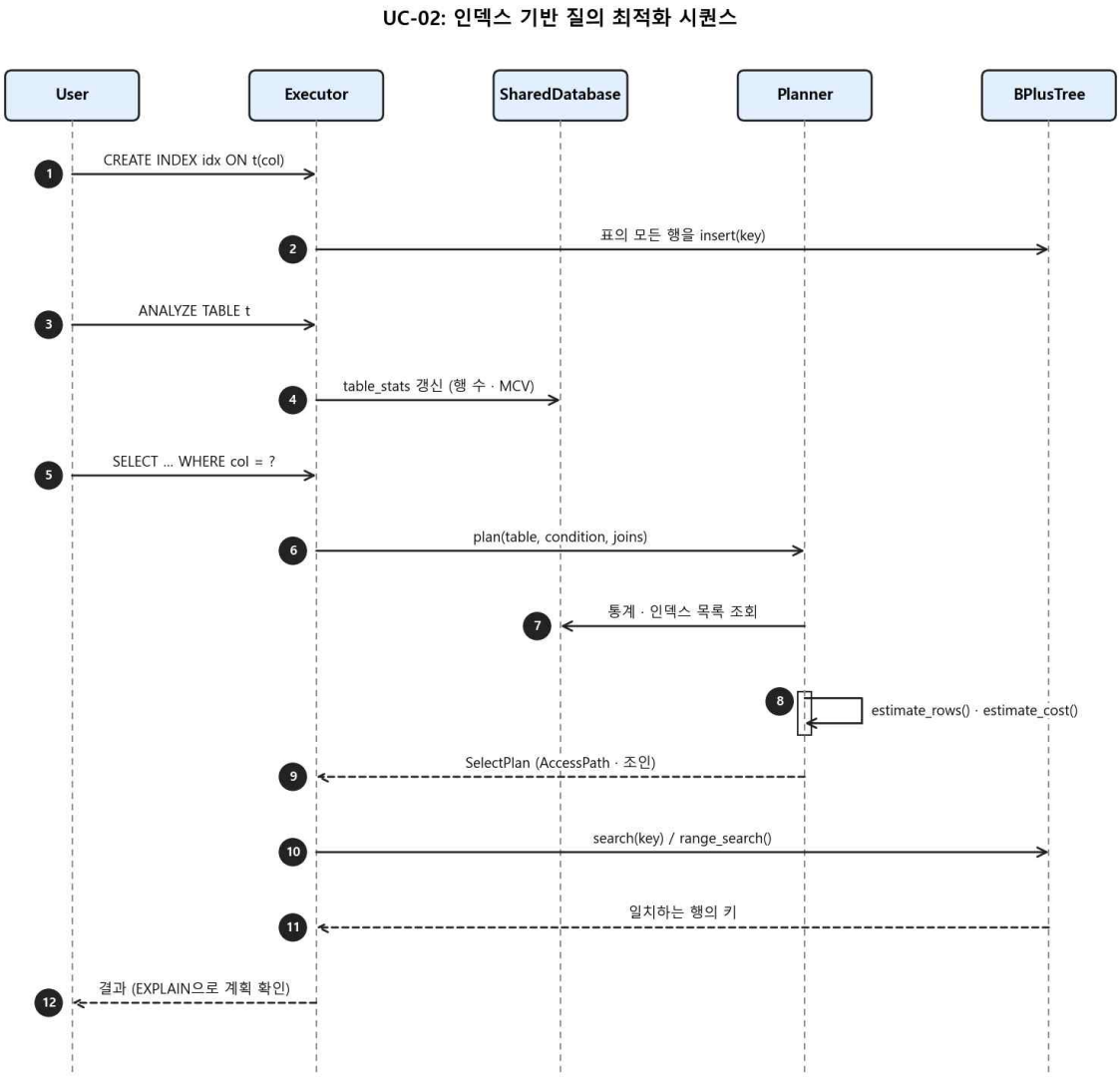


그림 3. choose_access( ) 시퀀스

1. 질의의 표와 조건에서 사용할 수 있는 인덱스(PK, 보조, 해시, 복합)를 모은다.
2. ANALYZE TABLE이 저장한 통계(행 수, 서로 다른 값 수, 대표값 목록, 히스토그램)로 후보별 행 수를 추정한다.
3. 후보 AccessPath(전체 스캔, 인덱스 조회 등)마다 비용을 계산한다.
4. 비용이 가장 낮은 접근 경로를 계획에 넣는다. 조인이 있으면 조인 알고리즘과 순서도 같은 방식으로 비교한다.
5. Executor가 그 경로로 BPlusTree를 search() 또는 range_search()하고, 읽은 행을 MVCC 가시성과 나머지 조건으로 걸러 낸다.

[3] 메소드 : exec_commit( ) @ C-005 Executor

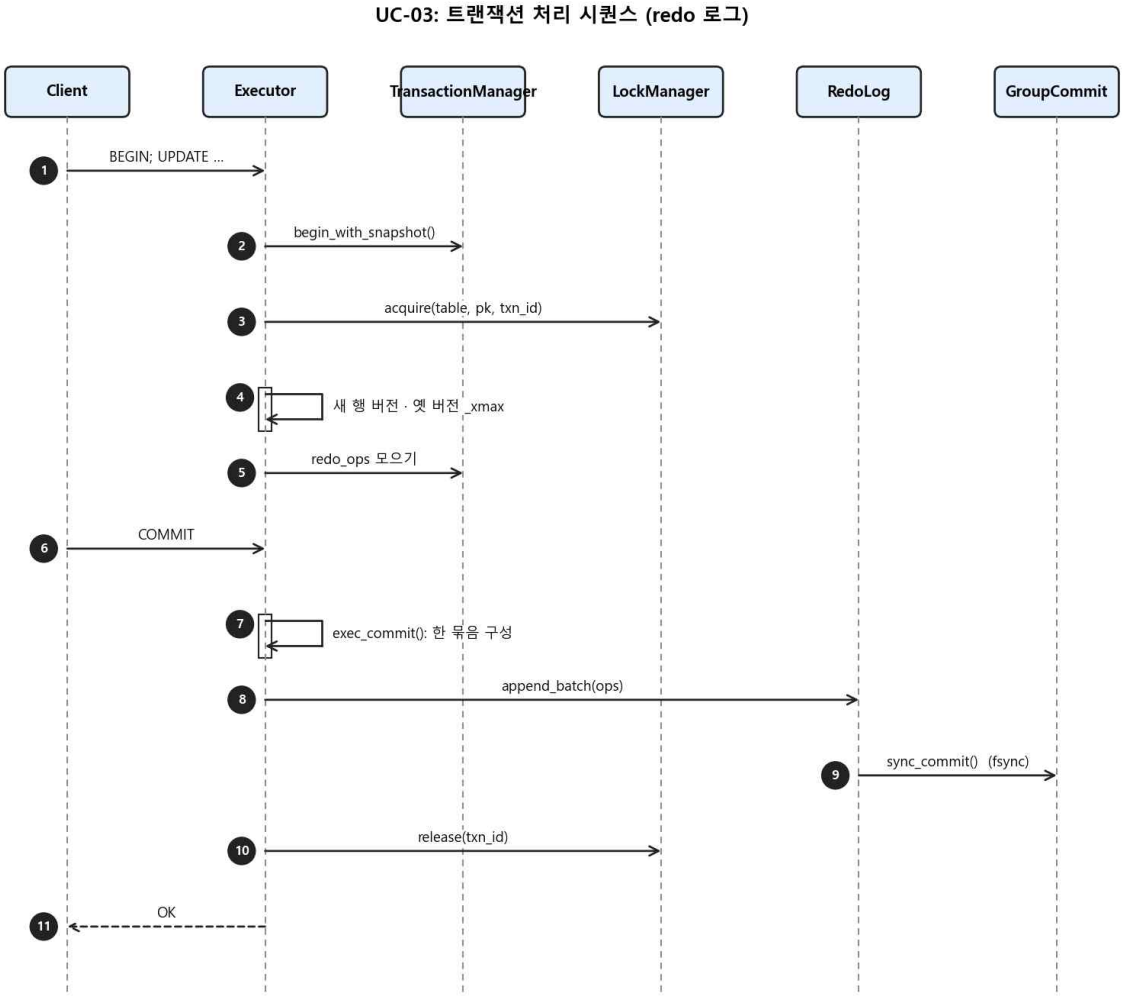


그림 4. exec_commit( ) 시퀀스

1. begin_with_snapshot()이 트랜잭션 번호를 주고 격리 수준에 맞는 스냅샷을 만든다. REPEATABLE READ와 SERIALIZABLE의 스냅샷은 BEGIN이 아니라 첫 번째 읽기(표를 읽는 SELECT)에서 뜬다.
2. 행을 바꿀 때마다 LockManager로 행 락을 얻고, 새 행 버전을 추가하면서 옛 버전에 _xmax를 표시한다.
3. 변경(InsertVersion, SetXmax)을 메모리에 모아 둔다.
4. COMMIT에서 변경을 체크섬이 붙은 묶음으로 만들어 RedoLog.append_batch()로 rusql.redo에 덧붙인다.
5. GroupCommitCoordinator.sync_commit()이 여러 세션의 COMMIT을 fsync 한 번으로 묶는다.
6. 락을 풀고 클라이언트에 OK를 돌려준다. 표 파일은 체크포인트에서만 다시 쓴다.

[4] 메소드 : recover_from_redo( ) @ C-005 Executor

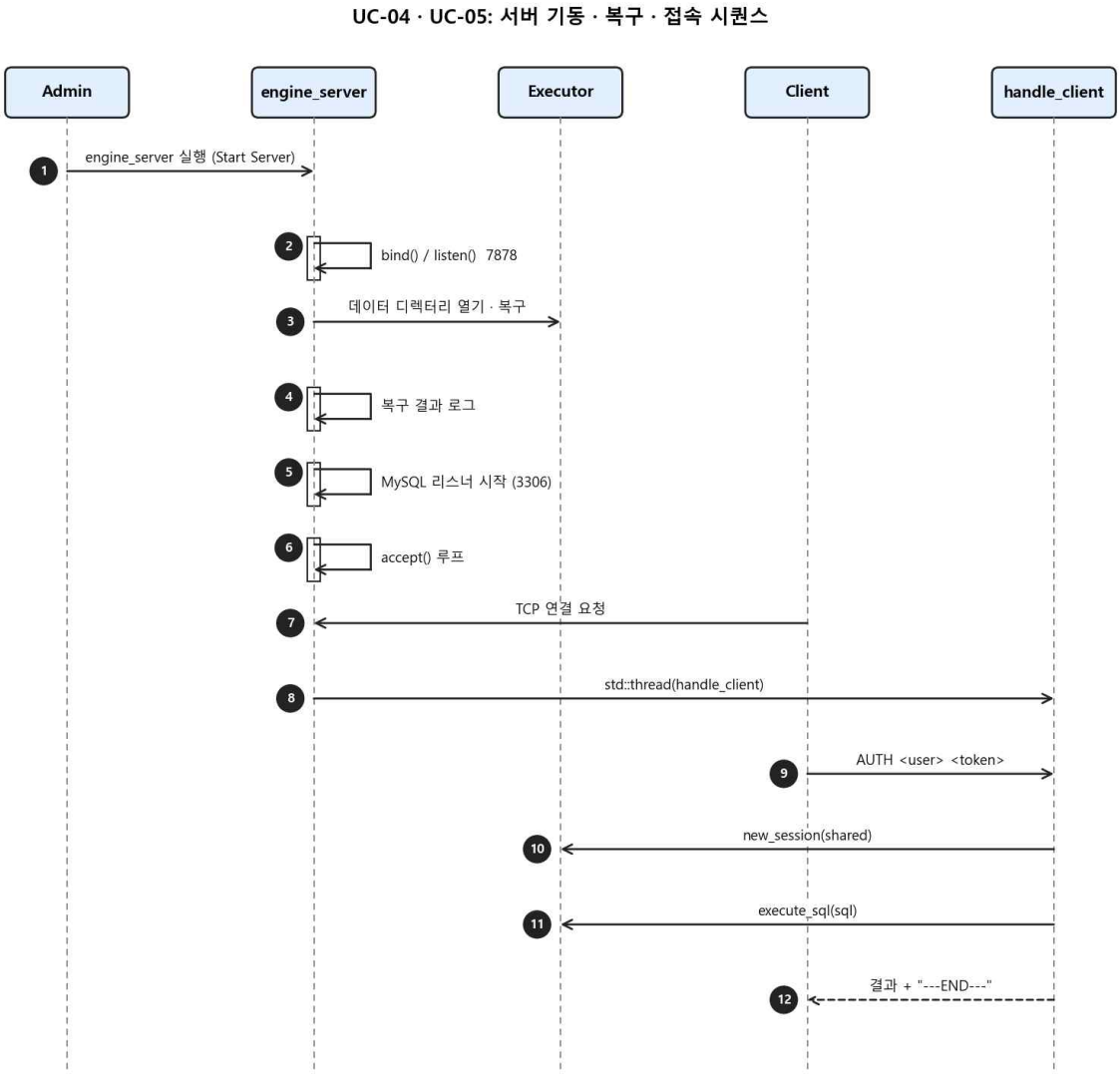


그림 5. recover_from_redo( ) 시퀀스

1. engine_server가 시작하면서 데이터 디렉터리를 열고 표와 스키마를 읽는다.
2. rusql.redo를 처음부터 읽어 체크섬이 맞는 묶음의 변경을 표에 다시 반영한다(커밋되지 않은 변경은 로그에 없다).
3. 인덱스를 다시 만들고 복구에 걸린 시간과 되살린 변경 수를 기록한다. Server Manager의 복구 카드가 이 기록을 보여 준다.
4. 복구가 끝나면 7878 포트와 MySQL 3306 포트를 열고 클라이언트 접속을 받는다.

3.3 DB 스키마 명세

RuSQL은 데이터베이스 엔진 자체이므로 응용 프로그램용 DB는 없다. 엔진이 내부에서 유지하는 시스템 카탈로그(TableSchema)와 느린 쿼리 로그 표를 표 2, 표 3과 같이 명세한다.

[1] 테이블 : TableSchema (시스템 카탈로그)

표 2. TableSchema (시스템 카탈로그)

Attributes	Types	설명 (Stored procedure 없음)
name{PK}	string	표 이름
columns	ColumnDef 목록	열 정의
primary_key_columns	string 목록	복합 기본키의 열 순서
check_constraints	CheckConstraint 목록	표 수준 CHECK
unique_keys	UniqueKey 목록	여러 열 UNIQUE
auto_increment_counters	열 이름 → 정수 맵	AUTO_INCREMENT 다음 값
partition_info	PartitionBy (선택)	파티션 정의

[2] 테이블 : information_schema.slow_log (느린 쿼리 로그, 최근 1000건)

표 3. information_schema.slow_log (느린 쿼리 로그, 최근 1000건)

Attributes	Types	설명 (Stored procedure 없음)
START_TIME	문자열 (YYYY-MM-DD HH:MM:SS.ffffff)	문장 시작 시각
USER_HOST	문자열	사용자와 접속 위치
QUERY_TIME	숫자 (초)	long_query_time 이상 걸린 시간
ROWS_SENT	숫자	돌려보낸 행 수
DB	문자열	사용 중인 데이터베이스
SQL_TEXT	문자열	문장 본문
THREAD_ID	숫자	세션 번호

3.4 API 명세

외부와 주고받는 인터페이스는 두 가지 접속 프로토콜과 AI 어시스턴트가 쓰는 MCP 도구 37종이다. 표 4에 모두 명세한다.

표 4. 외부 인터페이스(프로토콜과 MCP 도구) 명세

인터페이스명	클래스명	인터페이스 대상	설명
TCP 7878 (자체 프로토콜)	C-020 engine_server	rusql-ui, rusql-mcp, engine_client	AUTH 챌린지-응답으로 인증한 뒤 SQL을 보내고 결과를 "---END---"로 끝나는 텍스트로 받는다.

인터페이스명	클래스명	인터페이스 대상	설명
TCP 3306 (MySQL 와이어 프로토콜)	C-020 engine_server	mysql CLI, DBeaver 등	핸드셰이크, 인증, COM_* 명령(텍스트 · 준비된 문장, 준비된 문장의 결과는 바이너리 프로토콜 행)을 MySQL 8.4 클라이언트와 같은 방식으로 처리한다.
execute_sql(sql, database)	MCP 서버	RuSQL 서버	SQL을 실행하고 SELECT는 행 객체의 JSON 배열로 돌려준다. 위험한 SQL은 실행하지 않고 확인을 요구한다.
confirm_dangerous_sql(sql, database, via_editor, tab)	MCP 서버	RuSQL 서버	거절된 위험 SQL(DROP 등)을 사용자가 확인한 뒤 실행한다.
run_script(statements, database, commit, confirmed_dangerous)	MCP 서버	RuSQL 서버	여러 문장을 한 연결의 한 트랜잭션에서 실행하고 문장별 영향을 보고한다. 확인 전에는 커밋하지 않는다.
list_databases()	MCP 서버	RuSQL 서버	데이터베이스 목록을 JSON으로 돌려준다.
list_tables(database)	MCP 서버	RuSQL 서버	데이터베이스의 표 목록을 JSON으로 돌려준다.
get_table_schema(table, database)	MCP 서버	RuSQL 서버	표의 CREATE TABLE 문을 돌려준다.
explain_query(sql, database)	MCP 서버	RuSQL 서버	EXPLAIN ANALYZE를 실행하여 실행 계획을 구조화해 돌려준다.
get_indexes(table, database)	MCP 서버	RuSQL 서버	표에 정의된 인덱스 목록을 돌려준다.
suggest_indexes(sql, database, verify)	MCP 서버	RuSQL 서버	SELECT, UPDATE, DELETE에 필요한 CREATE INDEX를 제안한다. verify이면 임시로 만들어 전·후 시간을 재고 지운다.
list_locks()	MCP 서버	RuSQL 서버	열린 트랜잭션, 잡힌 행 락, 락 대기 관계를 보여 준다.
health_report()	MCP 서버	RuSQL 서버	가동 시간, 연결, 문장, 락, 교착, 복구 상태를 한 번에 요약한다.
get_table_stats(table, database)	MCP 서버	RuSQL 서버	행 수와 열별 통계(서로 다른 값 수, NULL 수 등)를 JSON으로 돌려준다.
get_fk_graph(database, table, to_table)	MCP 서버	RuSQL 서버	외래키 관계를 간선 목록으로 돌려준다. JOIN을 쓸 때 경로를 찾는 데 쓴다.

인터페이스명	클래스명	인터페이스 대상	설명
trace_query(sql, database)	MCP 서버	RuSQL 서버	문장이 엔진의 단계(파싱, 준비, 락, 스캔, 조인 등)를 지나는 시간을 단계별로 돌려준다.
sample_data(table, n, database)	MCP 서버	RuSQL 서버	표에서 최대 n개의 행을 샘플로 돌려준다.
slow_queries(limit, min_seconds)	MCP 서버	RuSQL 서버	느린 쿼리 로그의 문장을 문장 본문별로 묶어 횟수와 걸린 시간 순으로 돌려준다. 로그가 꺼져 있으면 켜는 방법을 알려 준다.
kill_session(session_id, kill_connection, confirmed)	MCP 서버	RuSQL 서버	멈춘 세션의 문장을 중단하거나(KILL QUERY) 연결을 끊는다(KILL CONNECTION). confirmed 전에는 세션 내용만 알려 주고 거절한다.
import_csv(table, file_path, database, has_header, delimiter, empty_as_null, commit)	MCP 서버	RuSQL 서버	CSV 파일의 행을 한 트랜잭션으로 표에 넣는다. commit 전에는 미리보기로 실행해 되돌리고 오류만 보여 준다.
export_csv(sql, file_path, database, overwrite)	MCP 서버	RuSQL 서버	SELECT 결과를 CSV 파일로 쓴다(최대 100만 행). 이미 있는 파일은 overwrite일 때만 덮어쓴다.
diff_schema(database_a, database_b)	MCP 서버	RuSQL 서버	두 데이터베이스의 표 · 열 · 인덱스 · 외래 키 차이를 돌려준다. 아무것도 바꾸지 않는다.
list_connections()	MCP 서버	관리 도구	관리 도구 홈 화면에 저장된 연결 목록을 돌려준다.
add_connection(name, host, port, user, password)	MCP 서버	관리 도구	홈 화면에 새 연결을 추가한다.
delete_connection(id_or_name)	MCP 서버	관리 도구	저장된 연결을 삭제한다.
update_connection(id_or_name, name, host, port, user, password, auto_login)	MCP 서버	관리 도구	저장된 연결의 값을 바꾼다.
write_to_editor(query, tab, instance)	MCP 서버	관리 도구	편집기 탭의 내용을 주어진 SQL로 바꾼다.

인터페이스명	클래스명	인터페이스 대상	설명
new_editor_tab(name, query, instance)	MCP 서버	관리 도구	새 편집기 탭을 열고 SQL을 채울 수 있다.
close_editor_tab(tab, instance)	MCP 서버	관리 도구	이름이 같은 편집기 탭을 닫는다.
switch_editor_tab(tab, instance)	MCP 서버	관리 도구	이름이 같은 편집기 탭으로 이동한다.
list_editor_tabs(instance)	MCP 서버	관리 도구	열려 있는 편집기 탭 이름을 순서대로 돌려준다.
get_editor_tab_content(tab, instance)	MCP 서버	관리 도구	편집기 탭의 현재 SQL 텍스트를 돌려준다.
execute_in_editor(query, tab, timeout_seconds, instance)	MCP 서버	관리 도구	사용자가 Run을 누른 것처럼 편집기에서 SQL을 실행하고 화면에 결과를 보여 준다.
list_app_instances()	MCP 서버	관리 도구	실행 중인 RuSQL 앱 창과 로그인 여부를 돌려준다.
launch_app()	MCP 서버	관리 도구	실행 중인 앱이 없으면 RuSQL 데스크톱 앱을 실행한다.
login(connection, instance)	MCP 서버	관리 도구	실행 중인 앱 창을 저장된 연결로 로그인시킨다.
start_server(port, mysql_port, instance)	MCP 서버	관리 도구	서버 매니저의 수신 포트(자체 프로토콜, MySQL 프로토콜)를 시작한다.
stop_server(instance)	MCP 서버	관리 도구	start_server로 시작한 수신 포트를 멈춘다.
get_server_status(instance)	MCP 서버	관리 도구	서버 매니저의 현재 상태를 돌려준다.

4. 사용자 인터페이스(GUI)

사용자가 시스템을 쓰는 순서에 따라 실제 실행 화면을 그림 6부터 그림 12로 나타낸다.

1) 홈 화면에서 저장된 연결을 골라 서버에 접속한다.

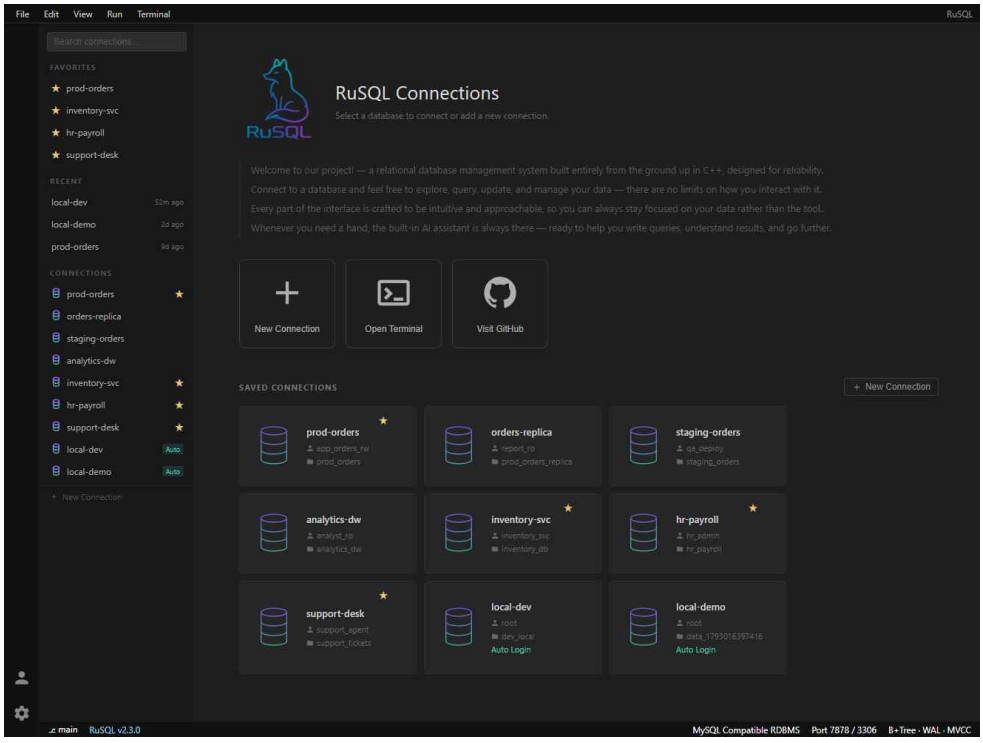


그림 6. 홈 화면

2) 편집기에서 SQL을 작성하고 Ctrl+Enter로 실행한다. 결과는 아래 표에 나온다.

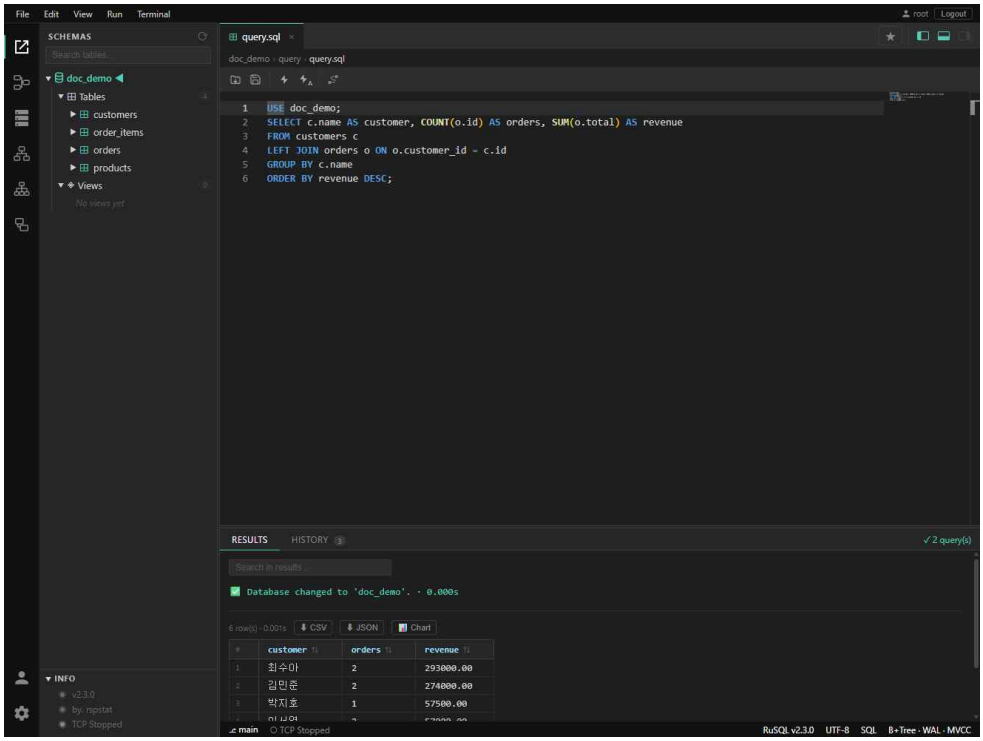


그림 7. 메인 에디터 뷰 (사이드바, 에디터, 결과)

3) EXPLAIN으로 실행 계획을 그림으로 보고, 인덱스 제안을 받는다.

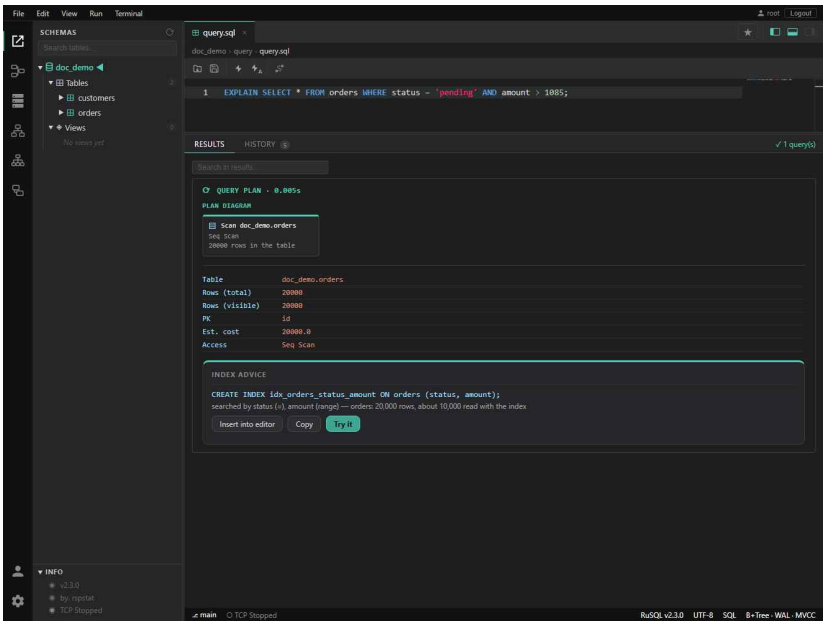


그림 8. 실행 계획과 인덱스 제안

4) ERD Editor에서 표와 외래키 관계를 보고 편집한다.



그림 9. ERD Editor 뷰

5) 서버 관리 화면에서 서버를 시작·중지하고 활동 로그를 본다.

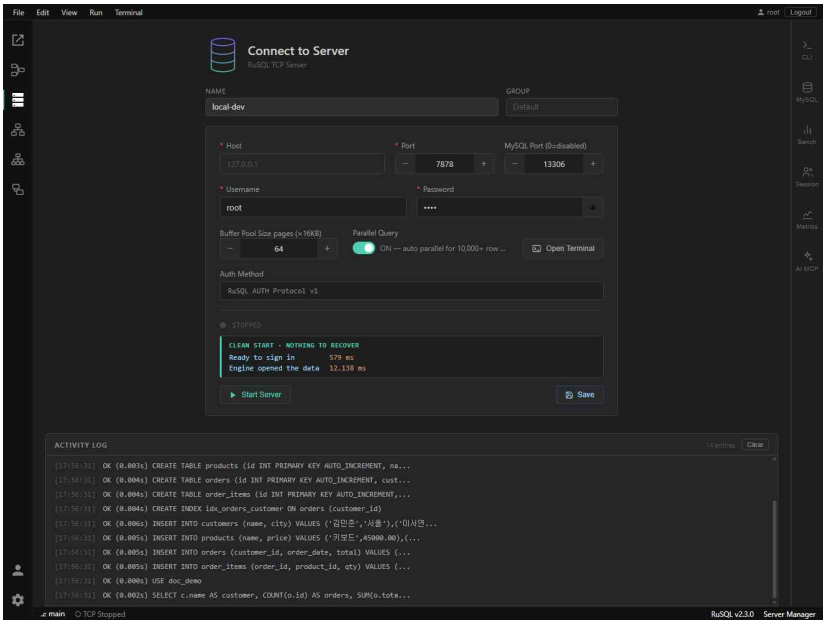


그림 10. 서버 관리 뷰

6) 세션·트랜잭션·락 패널에서 실행 중인 세션과 락을 보고 필요하면 KILL 한다.

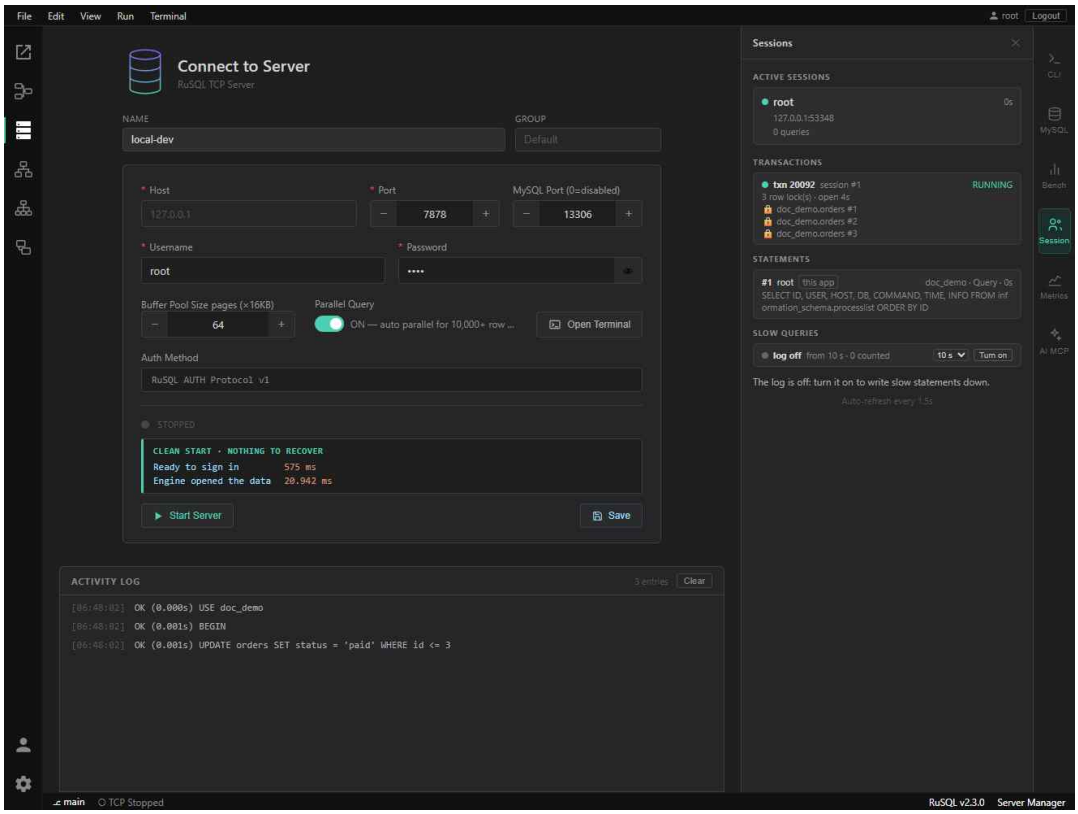


그림 11. 세션·트랜잭션·락 패널

7) AI MCP 패널에서 Claude Desktop 연결을 자동으로 구성한다.

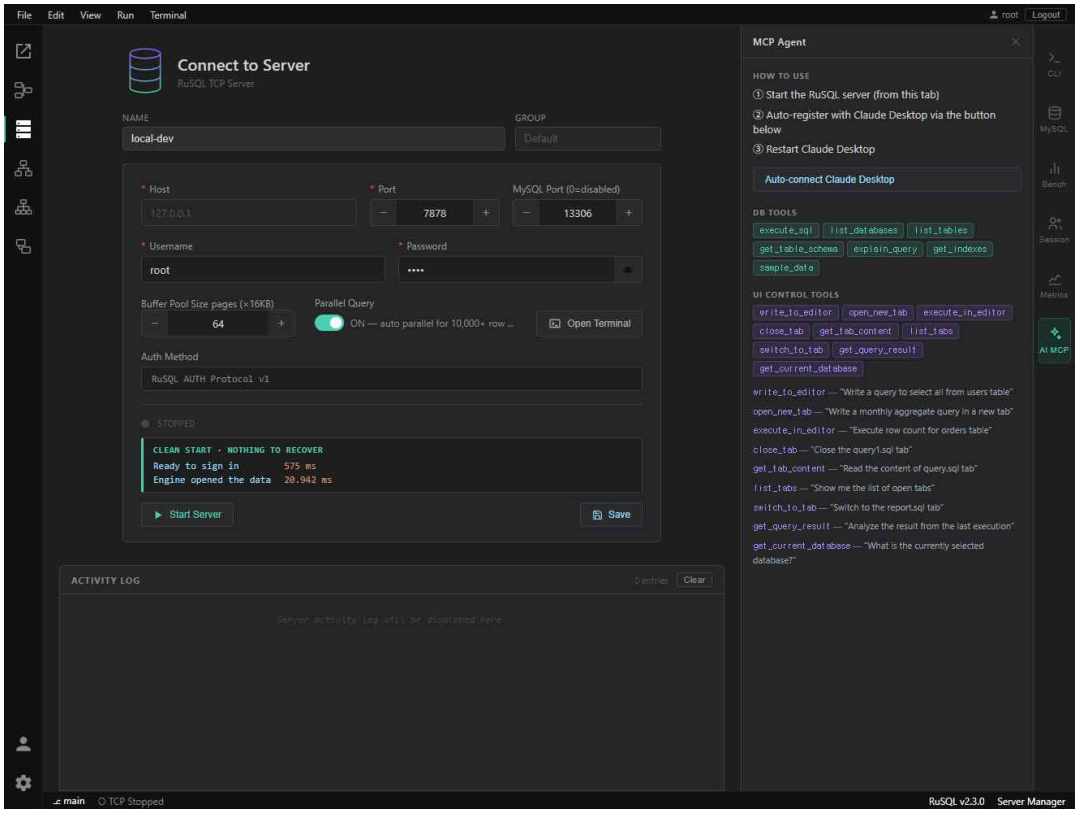


그림 12. AI MCP 패널

5. 기타 참고 사항

1. 오픈소스 사용 정보 : C++ 엔진은 nlohmann/json 3.11.3(MIT), LZ4 1.9.4(BSD-2-Clause), Catch2 2.13.10(BSL-1.0), PicoSHA2(MIT), SHA-1 참고 구현(퍼블릭 도메인)을 쓴다. 데스크톱 도구는 Tauri 2(MIT/Apache-2.0), React 19(MIT), Monaco Editor(MIT), Vite(MIT)를, MCP 서버는 Python MCP SDK(MIT)를 쓴다.
2. 재사용한 소스 : 1학기에 Rust로 구현한 엔진(legacy 브랜치)을 2026-07-07부터 07-08까지 C++로 옮겼다. 처음에는 동작을 그대로 옮기는 1:1 포팅으로 시작하고 이후에 재설계하였다.
3. 개발 범위 : XA 분산 트랜잭션, 페이지 단위 버퍼 풀 교체 알고리즘, WAL 기반 복제(replication)는 2026-08-09 범위 검토에 따라 제외하였다.
4. 다음 단계에 고려할 사항 : 남은 기간(2026-12-31까지)에는 새 기능 대신 정합성과 안정성 보강, 문서 정리를 진행한다. 알려진 한계는 쓰는 문장 안의 상관 서브쿼리와 NATURAL·USING 조인이 아직 중첩 루프라는 점, 열 단위 COLLATE·BINARY가 없다는 점이다.

<끝>

2026년 충북대학교 소프트웨어학부 졸업작품연구과제

[RuSQL]

소프트웨어 시스템 테스트 결과서

2026년 11월 9일

문서번호 : 26-RuSQL-Doc-007

소 속 : 충북대학교 소프트웨어학과

팀 명 : RuSQL

팀 원 : 김규현, 빌궁

책임교수 : 이종연 교수님

목 차

1. 프로젝트 개요 ---- 1
1.1 시스템 명 ---- 1
1.2 시스템 개요 ---- 1

2. 테스트 개요 ---- 1
2.1 테스트 범위 ---- 1
2.2 테스트 항목 및 통과 기준 ---- 1

3. 테스트 케이스 ---- 4

4. 테스트 예외사항 ---- 8
4.1 중단 기준과 재개 조건 ---- 8

5. 테스트 환경 ---- 8
5.1 HW 테스트 환경 구성 ---- 8
5.2 SW 테스트 환경 구성 ---- 9

6. 테스트 결과 ---- 10

7. 부록 ---- 15

1. 프로젝트 개요

1.1 시스템명

1. 국문 : AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)
2. 영문 : AI/MCP Integrated Relational Database Management System (RuSQL)

1.2 시스템 개요

이 작품은 관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션/동시성 제어, 서버 등)을 구현하고, 이를 다루는 데스크톱 관리 도구와 AI/MCP 기술의 연동 기능을 갖춘 MySQL 프로토콜과 호환되는 관계형 데이터베이스 시스템이다.

2. 테스트 개요

2.1 테스트 범위

1. 본 시스템 테스트는 D04 프로덕트 백로그의 주제 6개(T-01 ~ T-06)와 에픽 12개(E-01 ~ E-12) 전체를 대상으로 한다.
2. 엔진(E-01 ~ E-06)은 자동 회귀 테스트(Catch2, 946개 케이스)와 실제 서버에 SQL을 보내는 시나리오 테스트로 검증한다.
3. MCP 도구(E-10, E-12)는 실제 서버에 도구를 호출하는 시나리오로 검증한다. 관리 도구 화면(E-07 ~ E-09)과 AI의 화면 제어(E-11)는 앱에서 직접 수행하는 수동 시나리오로 검증한다.
4. 2026-08-09 범위 검토에 따라 개발 범위에서 제외한 XA 분산 트랜잭션, 페이지 단위 버퍼 풀 교체 알고리즘, WAL 기반 복제(replication)는 테스트 대상에서도 제외한다.
5. Claude Desktop 자체의 응답 품질은 이 시스템의 범위가 아니므로 테스트하지 않고, MCP 도구가 올바른 결과를 돌려주는지만 확인한다.

2.2 테스트 항목 및 통과 기준

이 항목에서는 테스트를 실시하는 대상과 통과 기준을 표 1과 같이 정의한다. 기능구현범위는 명세된 기능 중 구현을 확인한 비율이고, 실패한 테스트 케이스가 없어야 통과한다.

표 1. 테스트 항목 및 통과 기준

테스트 항목		척도	통과기준
T-01 SQL 처리	*E-01 SQL 파싱·실행*	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
	E-02 비용 기반 질의 최적화	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
T-02 트랜잭션·동시성	*E-03 MVCC 트랜잭션 처리*	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
	E-04 락·데드락 감지	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
T-03 장애 복구	*E-05 WAL·Undo 복구*	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
T-04 MySQL 프로토콜 호환	*E-06 MySQL 와이어 프로토콜 서버*	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
T-05 관리 도구	*E-07 SQL 편집기*	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케	*0.0*

테스트 항목		척도	통과기준
		이스의 수 = Y개 *테스트 종료 조건 = Y/X*	
	E-08 스키마·ERD 뷰어	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
	E-09 서버 모니터링	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
T-06 MCP·AI 연동	*E-10 MCP로 DB 제어*	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
	E-11 MCP로 관리 도구 제어	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*
	E-12 위험 SQL 안전장치	*기능구현범위 : X = A/B* *A : 구현확인된 기능 수, B : 명세된 기능 수*	*1.0*
		실행된 테스트 케이스의 수 = X 개, 실패한 테스트 케이스의 수 = Y개 *테스트 종료 조건 = Y/X*	*0.0*

3. 테스트 케이스

테스트 케이스를 요구사항 그룹별로 두 표로 나눈다. 표 2는 엔진(E-01 ~ E-06), 표 3은 MCP 도구와 관리 도구(E-07 ~ E-12)의 케이스이다. 케이스마다 ID(STC-에픽-번호)를 두었고, 정상 동작과 비정상 동작(잘못된 SQL, 교착, 접근 거부, 위험 SQL)을 모두 포함한다.
입력 데이터는 시험에 실제로 쓴 SQL과 값이다. shop 데이터베이스에 emp(id, name, dept), dept(id, dname), big(id, k, s), ledger(id, amt) 표를 만들어 사용하였다.

표 2. 엔진 테스트 케이스

Test ID	테스트 항목	입력 데이터/시나리오	출력 데이터
STC-E01-001	E-01 US-01	CREATE TABLE emp (id INT PRIMARY KEY, name VARCHAR(20), dept INT); INSERT INTO emp VALUES (1,'Kim',10),(2,'Lee',10),(3,'Park',20); SELECT * FROM emp ORDER BY id	3행 반환: (1,Kim,10) (2,Lee,10) (3,Park,20)
STC-E01-002	E-01 US-01	UPDATE emp SET dept = 30 WHERE id = 3; DELETE FROM emp WHERE id = 2; SELECT id, dept FROM emp ORDER BY id	(1,10) (3,30)
STC-E01-003	E-01 US-01	SELEC * FROM emp (잘못된 SQL), 이어서 SELECT 1	오류 메시지를 돌려주고 서버는 계속 동작한다. SELECT 1 = 1
STC-E01-004	E-01 US-02	CREATE TABLE dept (id INT PRIMARY KEY, dname VARCHAR(20)); INSERT INTO dept VALUES (10,'Dev'),(30,'Ops'); SELECT e.name, d.dname FROM emp e JOIN dept d ON e.dept = d.id ORDER BY e.id	(Kim,Dev) (Park,Ops)
STC-E01-005	E-01 US-02	SELECT name FROM emp WHERE dept IN (SELECT id FROM dept WHERE dname = 'Dev')	(Kim)
STC-E01-006	E-01 US-02	WITH t AS (SELECT dept, COUNT(*) AS c FROM emp GROUP BY dept) SELECT dept, c FROM t ORDER BY dept	(10,1) (30,1)

Test ID	테스트 항목	입력 데이터/시나리오	출력 데이터
STC-E02-001	E-02 US-03, US-04	big(id INT PRIMARY KEY, k INT, s VARCHAR(12)) 5,000행(k = id % 100)을 넣고 ANALYZE TABLE big; EXPLAIN SELECT * FROM big WHERE k = 7	k에 인덱스가 없으므로 Access: Seq Scan
STC-E02-002	E-02 US-03, US-04	CREATE INDEX idx_k ON big (k); ANALYZE TABLE big; EXPLAIN SELECT * FROM big WHERE k = 7	인덱스를 쓰는 계획, 예상 비용이 낮아진다
STC-E02-003	E-02 US-03	SELECT COUNT(*) FROM big WHERE k = 7	50 (인덱스를 써도 결과가 같다)
STC-E02-004	E-02 US-04	SUGGEST INDEX SELECT * FROM big WHERE s = 's77'	CREATE INDEX 문을 제안한다
STC-E03-001	E-03 US-05	세션 A: BEGIN; UPDATE emp SET dept = 99 WHERE id = 1 (커밋 전) / 세션 B: SELECT dept FROM emp WHERE id = 1 / A: COMMIT / B: 같은 SELECT	B는 커밋 전에는 10, 커밋 후에는 99를 읽는다
STC-E03-002	E-03 US-05	BEGIN; DELETE FROM emp; SELECT COUNT(*) FROM emp; ROLLBACK; SELECT COUNT(*) FROM emp	DELETE 직후 0, ROLLBACK 후 원래 행 수(2)
STC-E03-003	E-03 US-06	세션 B: SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ; BEGIN; SELECT dept (id = 1) / 세션 A: dept를 5로 UPDATE(자동 커밋) / B: 같은 SELECT / B: COMMIT 후 SELECT	트랜잭션 안에서는 같은 값(99)이 보이고, 끝난 뒤에 5가 보인다
STC-E03-004	E-03 US-06	SET SESSION TRANSACTION ISOLATION LEVEL SERIALIZABLE; SHOW ISOLATION LEVEL	격리 수준이 SERIALIZABLE로 바뀐다
STC-E04-001	E-04 US-07	세션 A: BEGIN; UPDATE emp SET dept = 1 WHERE id = 1 / 세션 B: UPDATE emp SET dept = 2 WHERE id = 1 / 1초 뒤 A: COMMIT	B의 UPDATE는 A가 COMMIT 할 때까지 기다린 뒤 끝나고, 최종 dept = 2
STC-E04-002	E-04 US-07	세션 A: id 7 UPDATE, 세션 B: id 8 UPDATE / A가 id 8을, B가 id 7을 UPDATE (서로 기다림)	한 트랜잭션이 교착으로 중단되고 다른 하나는 진행한다

Test ID	테스트 항목	입력 데이터/시나리오	출력 데이터
STC-E04-003	E-04 US-07	SELECT COUNT(*) FROM big a, big b, big c WHERE a.k + b.k + c.k > 1 (긴 문장) 실행 0.8초 뒤 다른 세션에서 KILL QUERY <세션 번호>	긴 문장이 중단되고 오류가 돌아온다
STC-E05-001	E-05 US-08	ledger(id, amt)에 (1,100),(2,200)을 커밋 / 트랜잭션 안에서 (3,300) INSERT와 id 1의 UPDATE(999)를 커밋하지 않은 채 서버 프로세스를 강제 종료 / 서버 재기동 후 SELECT * FROM ledger	(1,100) (2,200): 커밋한 변경은 남고 커밋하지 않은 변경은 없다
STC-E05-002	E-05 US-08	재기동 직후 SELECT * FROM information_schema.recovery	크래시를 감지하고 되돌린 트랜잭션과 다시 반영한 변경 수를 보여 준다
STC-E05-003	E-05 US-08	복구 뒤 INSERT INTO ledger VALUES (4,400); SELECT COUNT(*) FROM ledger	3 (복구 뒤에도 쓰기가 정상)
STC-E06-001	E-06 US-09	mysql 8.4.9 클라이언트로 MySQL 포트에 접속하여 SELECT VERSION()	접속에 성공하고 8.4.9를 돌려준다
STC-E06-002	E-06 US-09	mysql 클라이언트: SELECT id, name FROM emp ORDER BY id (emp는 id 1, 3, 7, 8의 4행)	4행 반환
STC-E06-003	E-06 US-09	잘못된 비밀번호로 mysql 클라이언트 접속	접속이 거절된다

표 3. MCP 도구와 관리 도구 테스트 케이스

Test ID	테스트 항목	입력 데이터/시나리오	출력 데이터
STC-E10-001	E-10 US-13	MCP execute_sql('SELECT COUNT(*) AS n FROM emp', 'shop')	JSON 배열로 결과를 돌려준다
STC-E10-002	E-10 US-13	MCP list_tables('shop'), get_table_schema('emp', 'shop')	표 목록(JSON)과 emp의 CREATE TABLE 문
STC-E10-003	E-10 US-13	MCP explain_query('SELECT * FROM big WHERE k = 7', 'shop')	EXPLAIN ANALYZE 계획
STC-E12-001	E-12 US-15	MCP execute_sql('DROP TABLE dept', 'shop')	실행하지 않고 확인을 요구한다. dept 표는 그대로 있다

Test ID	테스트 항목	입력 데이터/시나리오	출력 데이터
STC-E12-002	E-12 US-15	MCP confirm_dangerous_sql('DROP TABLE dept', 'shop')	확인 뒤에야 DROP이 실행된다
STC-E07-001	E-07 US-10	관리 도구 편집기에 SELECT 문을 쓰고 Ctrl+Enter	결과 그리드에 행이 표시된다
STC-E07-002	E-07 US-10	편집기에서 EXPLAIN SELECT … 실행	계획 그림(상자와 화살표)과 인덱스 제안 카드가 나온다
STC-E08-001	E-08 US-11	ERD Editor 탭으로 이동	표 카드와 외래키 관계선이 그려진다
STC-E08-002	E-08 US-11	열을 끌어 다른 표의 열에 놓고 DDL 미리보기를 확인한 뒤 적용	ALTER TABLE … ADD FOREIGN KEY가 실행되고 관계선이 추가된다
STC-E09-001	E-09 US-12	서버 관리 화면에서 Start Server를 누르고 세션 패널을 연다	서버가 시작되고 접속한 세션이 표시된다
STC-E09-002	E-09 US-12	세션 패널에서 실행 중인 세션의 KILL을 누른다	해당 세션의 문장이 중단된다
STC-E11-001	E-11 US-14	Claude Desktop이 new_editor_tab, write_to_editor, execute_in_editor를 호출	편집기에 새 탭이 열리고 SQL이 채워져 실행된다

4. 테스트 예외사항

4.1 중단 기준과 재개 조건

테스트 도중 테스트를 중단해야 할 상황과 재개할 조건을 표 4와 같이 정의한다.

표 4. 중단 기준과 재개 조건

중단 기준	재개 조건
서버 프로세스가 비정상 종료(크래시)하여 이후 테스트를 진행할 수 없는 경우	로그로 원인을 확인하고 결함을 고친 뒤 서버를 다시 시작하여 중단된 테스트 케이스부터 재개한다.
테스트 중 데이터 파일이 손상되어 이후 결과 검증이 무의미해지는 경우	시험용 데이터 디렉터리를 초기화하고 해당 시나리오를 처음부터 다시 수행한다.
자동 회귀 테스트(Catch2)가 실패하는 경우	실패한 테스트 케이스를 먼저 고치고 전체가 통과함을 확인한 뒤 시스템 테스트를 재개한다.

5. 테스트 환경

5.1 HW 테스트 환경 구성

표 5는 시험에 쓴 하드웨어이다. 서버, 관리 도구, 클라이언트, MCP 서버를 한 대의 PC에서 함께 실행하였다.

표 5. HW 테스트 환경

역할	H/W 제품내역		수량
	제품명	세부내역	
서버 + 클라이언트	데스크톱 PC	– CPU : AMD Ryzen 5 5600X (6코어 12스레드) – Memory : 16 GB – Local Disk : SSD 512 GB, HDD 2 TB	1

5.2 SW 테스트 환경 구성

표 6은 시험에 쓴 소프트웨어이다.

표 6. SW 테스트 환경

S/W		
구분	**구성요소**	**제품내역**
서버 공통	O/S	Windows 11 Pro 10.0.26300 (64비트)
	네트워크	TCP/IP (자체 프로토콜 7878, MySQL 프로토콜)
DB 서버	DBMS	RuSQL (자체 구현, engine_server.exe Release 빌드)
	테스트 프레임워크	Catch2 2.13.10
	비교 DBMS	MySQL Server 8.4.9
AI 연동	MCP 서버	Python 3.14.4, MCP SDK
	AI 클라이언트	Claude Desktop
클라이언트	접속 도구	mysql CLI 8.4.9, DBeaver
	관리 도구	RuSQL 데스크톱 앱 (Tauri 2, React 19)
빌드	컴파일러	Visual Studio 2022 (MSVC)
	빌드 도구	CMake 4.3.3

6. 테스트 결과

표 7은 2026-11-05에 실제 서버(engine_server.exe, Release 빌드)에서 수행한 시험 결과이다. 자동으로 수행한 28건은 모두 통과(P)하였고, 앱에서 직접 수행하는 7건은 수행 결과를 비워 두었다. 그 뒤 바뀐 엔진은 아래 1~3의 2026-11-09 재확인으로 다시 검증하였다.

1. 자동 회귀 테스트(Catch2): 946개 테스트 케이스, 1,520,540개 검증문이 모두 통과하였다(실패 0건). 에픽별 케이스 수는 E-01 567, E-02 124, E-03 76, E-04 90, E-05 28, E-06 61이다(Y/X = 0.0).

2. MCP 도구 단위 테스트 57건과 프런트엔드 라이브러리 테스트 26건이 통과하였고, 프런트엔드 TypeScript 타입 검사에 오류가 없다.

3. 실제 MySQL 8.4.9와의 대조 검증(코퍼스 49종, 4,209문장)은 2026-11-09 기준 차이 0건이다.

표 7. 테스트 결과

ID	테스트 설명	입력 값/ 시나리오	예상결과	수행 여부	수행결과	결과	정정여부
STC-E01-001	표 생성, 행 삽입, 조회	CREATE TABLE emp, INSERT 3행, SELECT	3행 반환: (1,Kim,10) (2,Lee,10) (3,Park,20)	Y	3행 반환: (1,Kim,10) (2,Lee,10) (3,Park,20)	P	-
STC-E01-002	UPDATE와 DELETE	UPDATE id = 3, DELETE id = 2	(1,10) (3,30)	Y	(1,10) (3,30)	P	-
STC-E01-003	잘못된 SQL의 오류 처리	SELEC * FROM emp	오류 메시지를 돌려주고 서버는 계속 동작한다. SELECT 1 = 1	Y	ERR Unknown statement: SELEC, 이어서 SELECT 1 = 1	P	-
STC-E01-004	JOIN	emp JOIN dept ON dept = id	(Kim,Dev) (Park,Ops)	Y	(Kim,Dev) (Park,Ops)	P	-
STC-E01-005	서브쿼리	WHERE dept IN (SELECT ...)	(Kim)	Y	(Kim)	P	-
STC-E01-006	CTE와 GROUP BY	WITH t AS (... GROUP BY dept)	(10,1) (30,1)	Y	(10,1) (30,1)	P	-
STC-E02-001	인덱스 없는 열의 실행 계획	EXPLAIN ... WHERE k = 7 (5,000행)	k에 인덱스가 없으므로 Access: Seq Scan	Y	Access: Seq Scan, 예상 비용 5000.0	P	-

ID	테스트 설명	입력 값/ 시나리오	예상결과	수행 여부	수행결과	결과	정정여부
STC-E02-002	인덱스를 만든 뒤 실행 계획	CREATE INDEX idx_k, 같은 EXPLAIN	인덱스를 쓰는 계획, 예상 비용이 낮아진다	Y	Access: Index Scan big_idx_k (k = 7), 예상 비용 24.6	P	-
STC-E02-003	인덱스 사용 결과의 정확성	COUNT(*) WHERE k = 7	50 (인덱스를 써도 결과가 같다)	Y	50	P	-
STC-E02-004	인덱스 제안	SUGGEST INDEX … WHERE s = 's77'	CREATE INDEX 문을 제안한다	Y	CREATE INDEX idx_big_s ON big (s); 제안, 읽는 행 5000 → 1	P	-
STC-E03-001	커밋 전후의 가시성	A: UPDATE 뒤 COMMIT, B: SELECT	B는 커밋 전에는 10, 커밋 후에는 99를 읽는다	Y	B가 읽은 값: 커밋 전 10, 커밋 후 99	P	-
STC-E03-002	ROLLBACK	BEGIN; DELETE; ROLLBACK	DELETE 직후 0, ROLLBACK 후 원래 행 수(2)	Y	DELETE 직후 0, ROLLBACK 후 2	P	-
STC-E03-003	REPEATABLE READ 스냅샷	B의 트랜잭션 중 A가 UPDATE	트랜잭션 안에서는 같은 값(99)이 보이고, 끝난 뒤에 5가 보인다	Y	B가 읽은 값: 99, 99, (COMMIT 후) 5	P	-
STC-E03-004	격리 수준 지정	SET … ISOLATION LEVEL SERIALIZABLE	격리 수준이 SERIALIZABLE로 바뀐다	Y	"Isolation level set to SERIALIZABLE." 응답, SHOW ISOLATION LEVEL 이 SERIALIZABLE을 표시	P	-
STC-E04-001	행 락 대기	A가 잡은 행을 B가 UPDATE	B의 UPDATE는 A가 COMMIT 할 때까지 기다린 뒤 끝나고, 최종 dept = 2	Y	B는 1초 뒤에도 기다리다 A의 COMMIT 뒤에 끝남 (1.0초 대기), 최종 dept = 2	P	-

ID	테스트 설명	입력 값/ 시나리오	예상결과	수행 여부	수행결과	결과	정정여부
STC-E04-002	교착 감지	A와 B가 서로의 행을 UPDATE	한 트랜잭션이 교착으로 중단되고 다른 하나는 진행한다	Y	B가 "Deadlock detected: transaction 5055 waits for transaction 5054 ... Transaction 5055 aborted." 오류를 받음, A는 진행	P	-
STC-E04-003	실행 중인 문장 중단	KILL QUERY <세션 번호>	긴 문장이 중단되고 오류가 돌아온다	Y	문장을 시작한 지 1.2초 만에(KILL은 0.8초에 보냄) "ERROR 1317 (70100): Query execution was interrupted"로 중단	P	-
STC-E05-001	강제 종료 후 복구	커밋 2행, 미커밋 변경 뒤 프로세스 종료	(1,100) (2,200): 커밋한 변경은 남고 커밋하지 않은 변경은 없다	Y	(1,100) (2,200)	P	-
STC-E05-002	복구 기록 조회	information_schema.recovery	크래시를 감지하고 되돌린 트랜잭션과 다시 반영한 변경 수를 보여 준다	Y	CRASH_DETECTED = YES, WAL_TXNS_UNDONE = 1, REDO_OPS = 2, REDO_TABLES = 1, OPEN_MS = 73	P	-
STC-E05-003	복구 뒤 쓰기	INSERT, COUNT(*)	3 (복구 뒤에도 쓰기가 정상)	Y	3	P	-
STC-E06-001	MySQL 클라이언트 접속	mysql ... SELECT VERSION()	접속에 성공하고 8.4.9를 돌려준다	Y	8.4.9	P	-
STC-E06-002	MySQL 클라이언트 조회	SELECT id, name FROM emp	4행 반환	Y	(1,Kim) (3,Park) (7,x) (8,y)	P	-

ID	테스트 설명	입력 값/ 시나리오	예상결과	수행 여부	수행결과	결과	정정여부
STC-E06-003	잘못된 비밀번호	mysql, 틀린 비밀번호	접속이 거절된다	Y	ERROR 1045 (28000): Access denied for user 'root' (using password: YES)	P	-
STC-E10-001	MCP SQL 실행	execute_sql	JSON 배열로 결과를 돌려준다	Y	[{"n": "4"}]	P	-
STC-E10-002	MCP 표 목록·스키마	list_tables, get_table_schema	표 목록(JSON)과 emp의 CREATE TABLE 문	Y	big, dept, emp 등 표 목록과 emp의 CREATE TABLE 문을 돌려줌	P	-
STC-E10-003	MCP 실행 계획	explain_query	EXPLAIN ANALYZE 계획	Y	QUERY PLAN (ANALYZE) 표를 돌려줌	P	-
STC-E12-001	위험 SQL 거절	execute_sql DROP TABLE	실행하지 않고 확인을 요구한다. dept 표는 그대로 있다	Y	"Refused: DROP is irreversible and affects an entire table/database. Ask the user to explicitly confirm ... " 응답, dept 표 유지	P	-
STC-E12-002	위험 SQL 확인 실행	confirm_dangerous_sql	확인 뒤에야 DROP이 실행된다	Y	Table 'shop.dept' dropped. dept 표가 사라짐	P	-
STC-E07-001	편집기에서 SQL 실행	SELECT 작성, Ctrl+Enter	결과 그리드에 행이 표시된다				
STC-E07-002	실행 계획 그림	EXPLAIN SELECT ...	계획 그림(상자와 화살표)과 인덱스 제안 카드가 나온다				

ID	테스트 설명	입력 값/ 시나리오	예상결과	수행 여부	수행결과	결과	정정여부
STC-E08-001	ERD 표시	ERD Editor 탭	표 카드와 외래키 관계선이 그려진다				
STC-E08-002	ERD에서 외래키 추가	열 끌어 놓기, 미리보기, 적용	ALTER TABLE … ADD FOREIGN KEY가 실행되고 관계선이 추가된다				
STC-E09-001	서버 시작과 세션 패널	Start Server, 세션 패널	서버가 시작되고 접속한 세션이 표시된다				
STC-E09-002	세션 KILL	세션 패널의 KILL	해당 세션의 문장이 중단된다				
STC-E11-001	MCP로 편집기 제어	new_editor_tab, write_to_editor, execute_in_editor	편집기에 새 탭이 열리고 SQL이 채워져 실행된다				

7. 부록

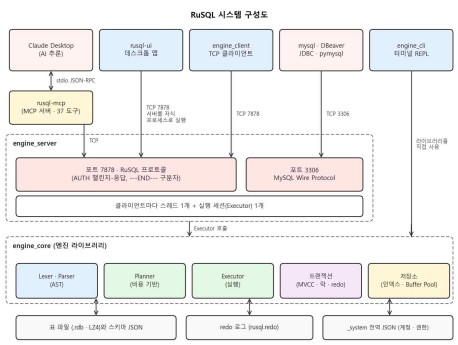
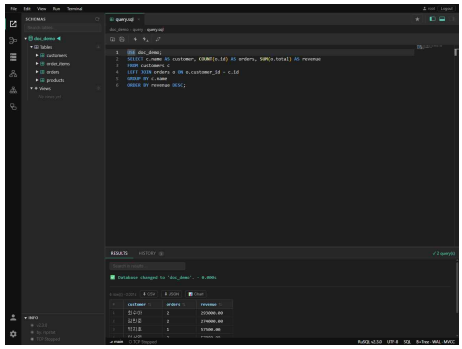
1. 자동 회귀 테스트 실행 방법 : cmake --build code/build --config Release --target engine_tests 로 빌드한 뒤 code/build/backend/tests/Release/engine_tests 를 실행한다.
2. 표 7의 시험 시나리오는 표 2와 표 3에 SQL과 값으로 기록하였다. MySQL 대조, 빌드 간 차분, 크래시 퍼저, 성능 측정 도구는 code/test 폴더에 있고 결과는 docs/mds의 SUMMARY.md와 DATE.md에 정리되어 있다.
3. 시험 중 발견한 사항 : CONNECTION_ID() 함수는 아직 지원하지 않는다. (SELECT @@transaction_isolation이 SET으로 바꾼 격리 수준을 돌려주지 않던 문제는 고쳤다.)

■

프로젝트 결과보고서

팀 번호(1 - 09)

<table>
<tr><td rowspan="2">프로젝트명
(주제)</td><td colspan="4">(국문) AI/MCP 통합 관계형 데이터베이스 관리 시스템 (RuSQL)</td></tr>
<tr><td colspan="4">(영문) AI/MCP Integrated Relational Database Management System (RuSQL)</td></tr>
<tr><td rowspan="2">팀장</td><td>학부(과)</td><td>소프트웨어학과</td><td>학번</td><td>2021041017</td></tr>
<tr><td>성 명</td><td>김규현</td><td>역할</td><td>팀장, 프로덕트 오너</td></tr>
<tr><td>개발기간</td><td colspan="4">2026년 3월 2일 ~ 2026년 12월 31일</td></tr>
<tr><td>GitHub저장소</td><td colspan="4">https://github.com/rspstat/RuSQL</td></tr>
<tr><td rowspan="4">참여학생</td><td>학부(과)</td><td>학년/학번</td><td>성명</td><td>역할</td></tr>
<tr><td>소프트웨어학과</td><td>2021041017</td><td>김규현</td><td>팀장, 시스템 설계·구현, 문서·저장소 관리</td></tr>
<tr><td>소프트웨어학과</td><td>2023041082</td><td>Namsraijalbuu Bilguuntuguldur</td><td>발표, AI 토큰 비용 관리, 산출물 문서 관리·검토</td></tr>
<tr><td></td><td></td><td></td><td></td></tr>
<tr><td>지도교수</td><td colspan="4">이종연</td></tr>
<tr><td colspan="5">프로젝트(주제) 수행 계획에 대한 요약</td></tr>
<tr><td>시스템
설명</td><td colspan="4">관계형 데이터베이스 엔진(SQL 처리, 저장, 트랜잭션·동시성 제어, 서버)을 직접 구현하고, 이를 다루는 데스크톱 관리 도구와 AI(MCP) 연동 기능을 갖춘 MySQL 프로토콜 호환 관계형 데이터베이스 시스템이다.</td></tr>
<tr><td>시스템
운영 환경</td><td colspan="4">Windows 11 PC 한 대에서 RuSQL 서버(TCP 7878, MySQL 프로토콜 3306), 데스크톱 관리 도구, MCP 서버를 함께 운영한다. 외부 클라이언트(mysql CLI, DBeaver)는 MySQL 프로토콜로, AI 어시스턴트(Claude Desktop)는 MCP 서버(도구 37종)로 접속한다.</td></tr>
<tr><td>개발 적용
기술</td><td colspan="4">C++ 엔진·서버(MSVC, CMake), Rust·Tauri와 React·TypeScript·Vite 데스크톱 도구, Python MCP 서버, Catch2 자동 회귀 테스트(946 케이스). 핵심 기법은 B+Tree·Hash 인덱스, 비용 기반 쿼리 최적화, 행 단위 MVCC와 4단계 격리 수준, 락·데드락 감지, WAL·redo 로그 복구, MySQL 와이어 프로토콜이다.</td></tr>
</table>

개발목표결과물 (시스템 구성도, 실행 화면)	  그림 1. 시스템 구성도(좌)와 관리 도구 실행 화면(우)
프로젝트 산출물 목록	1. 시스템 정의서(D01), 프로젝트 관리 계획서(D02), 소프트웨어 기능 아키텍처 (D03), 프로덕트 백로그(D04) 2. N차 스프린트 백로그(D05), N차 스프린트 명세서(D06) 3. 소프트웨어 시스템 시험 결과서(D07), 프로젝트 결과보고서(D08) 4. 소스 코드와 개발 문서(GitHub 저장소 rspstat/RuSQL, docs/mds)
시스템 활용 효과	1. 관계형 데이터베이스의 핵심 기법(MVCC, redo 로그 복구, 비용 기반 최적화)을 직접 구현하고 자동 회귀 테스트 946 케이스와 실제 MySQL 8.4.9 대조로 검증하여 동작 원리에 대한 이해를 높였다. 2. MCP 도구 37종으로 AI 어시스턴트가 자연어로 데이터베이스를 조회·진단하고 관리 도구를 조작할 수 있음을 확인했다(위험 SQL은 확인 후에만 실행). 3. MySQL 프로토콜을 지원하여 기존 도구(mysql CLI, DBeaver)로 바로 접속할 수 있다.
Key Words (5개) : 관계형 데이터베이스(RDBMS), MVCC, 비용 기반 쿼리 최적화, 트랜잭션, MCP	

작성자 성명 : 김규현 확인자 성명 :

부록: 최종 개발된 소프트웨어 세부 내용 작성

1. 최종 시스템 아키텍처 (System Architecture)

1.1 최종 시스템 아키텍처 개요

시스템 구성은 그림 2, 그림 3과 같다. RuSQL 서버(engine_server)는 자체 프로토콜(TCP 7878)과 MySQL 프로토콜(TCP 3306)로 접속을 받고, 클라이언트마다 스레드와 실행 세션(Executor)을 하나씩 둔다. 관리 도구(rusql-ui)와 MCP 서버(rusql-mcp)는 같은 서버에 접속하고, AI 어시스턴트는 MCP 서버를 통해서만 서버와 관리 도구를 다룬다. 데이터는 로컬 디스크의 표 파일, 스키마 JSON, redo 로그(rusql.redo)에 저장한다.

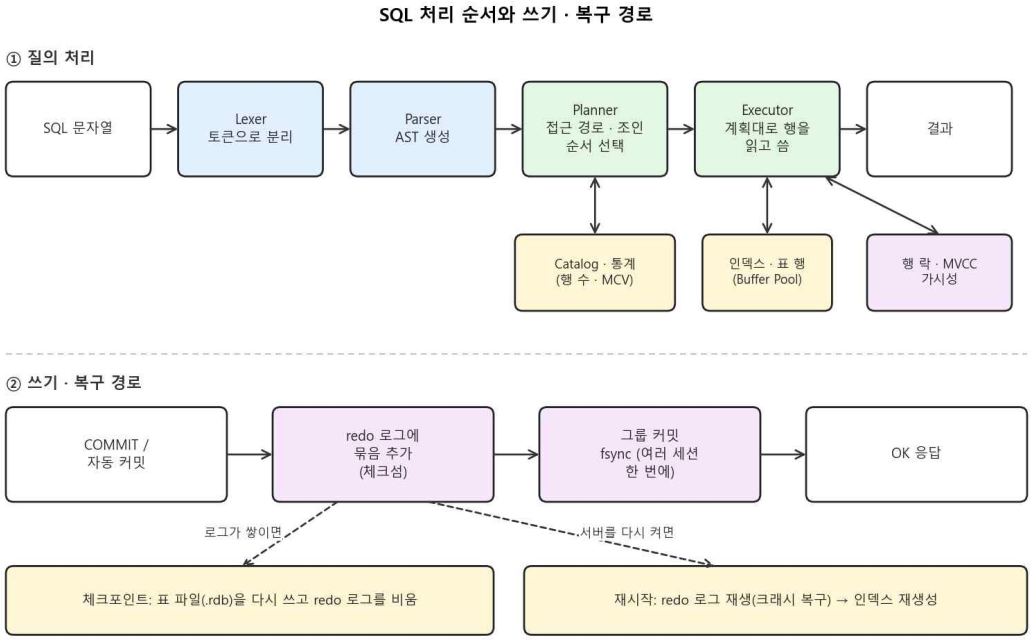


그림 2. SQL 처리 순서와 쓰기 · 복구 경로

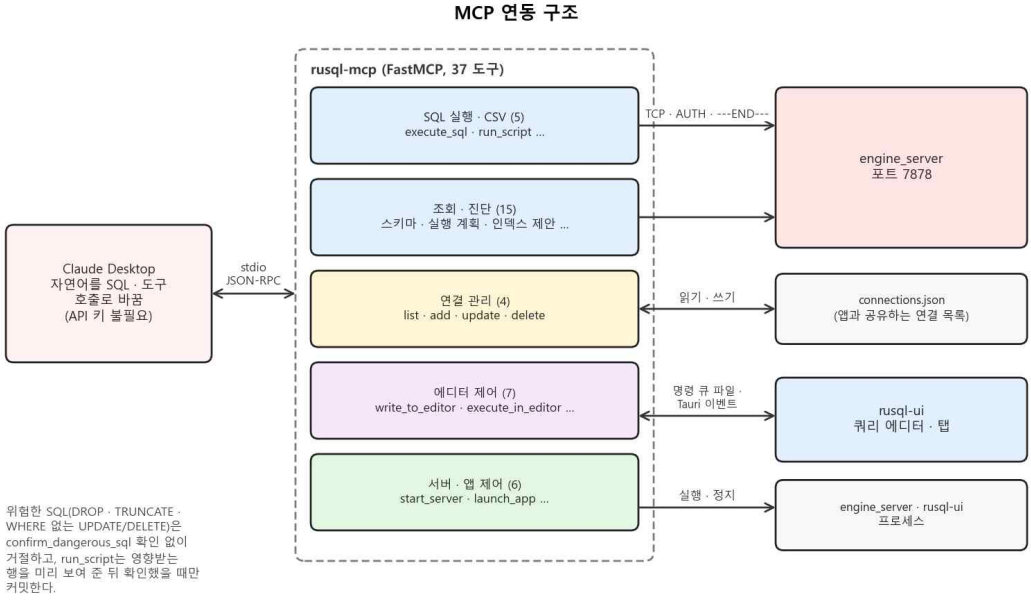


그림 3. MCP 연동 구조

2. 소프트웨어 아키텍처 (Software Architecture)

소프트웨어는 파서, 실행, 계획 등, 카탈로그, 저장소, 트랜잭션, 서버·클라이언트, 관리 도구(Frontend), MCP의 9개 컴포넌트로 나눈다. 계층은 서버 → 엔진 → 파서 · 카탈로그 · 저장소 · 트랜잭션 순이다.

2.1 아키텍처 구조

그림 4는 컴포넌트·패키지 사이의 의존 관계(점선 화살표 «use»)와 각 패키지에 속한 주요 클래스를 나타낸다.

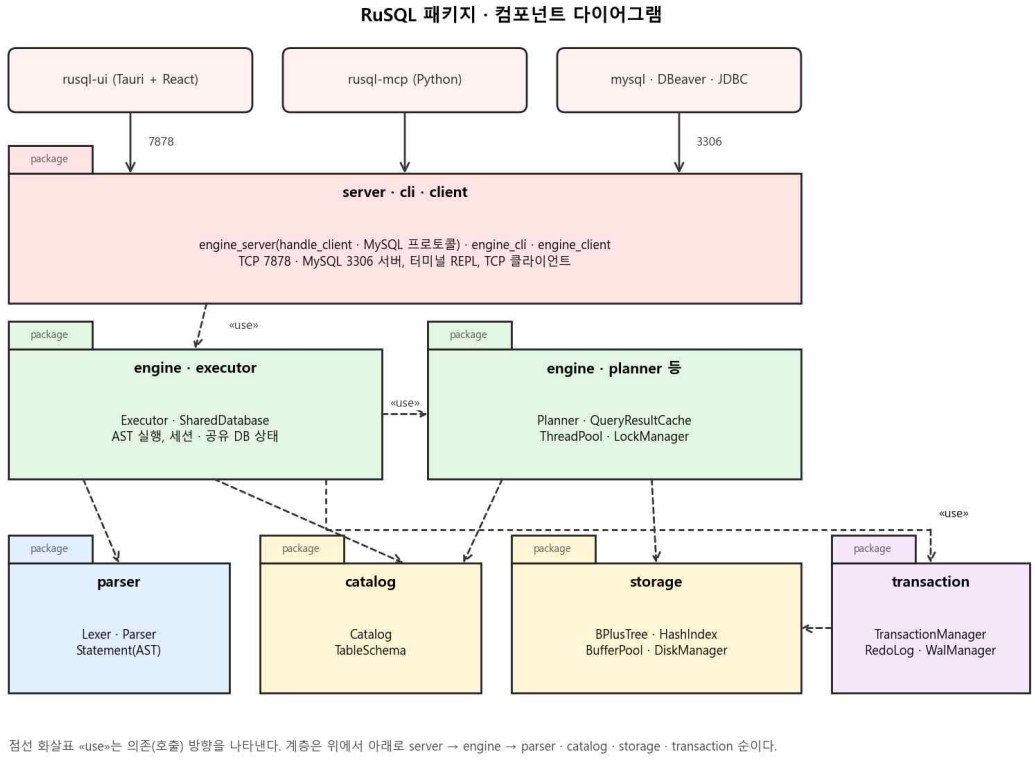


그림 4. 패키지 · 컴포넌트 다이어그램

2.2 컴포넌드/패키지 설명

표 1은 2.1절의 컴포넌트·패키지를 설명한다.

번호	컴포넌트/패키지 이름	설명	비고
1	Parser	토큰화와 구문 분석으로 SQL을 AST로 바꾼다.	
2	Executor	AST를 실행하고 세션과 공유 DB 상태를 관리한다.	
3	Planner 등	비용 기반 계획, 결과 캐시, 병렬 실행, 행 단위 락·데드락 감지.	
4	Catalog	표·열·제약·파티션 메타데이터.	
5	Storage	B+Tree·Hash 인덱스, 페이지 캐시, 표 파일 입출력.	
6	Transaction	MVCC, redo 로그, 그룹 커밋, 크래시 복구.	
7	Server · CLI · Client	TCP 7878과 MySQL 3306 서버, 터미널 REPL, TCP 클라이언트.	C++
8	Frontend (rusql-ui)	Tauri + React 데스크톱 관리 도구.	Rust, TypeScript
9	MCP (rusql-mcp)	AI 어시스턴트가 서버와 관리 도구를 다루는 도구 37종.	Python

3. 모듈 명세 (Module Specification)

2.2절의 컴포넌트·패키지에 포함된 클래스를 간단히 명세한다. 클래스의 멤버 변수와 전체 메서드는 D06의 3.1절에 있다. 비고 열의 C-번호는 D06의 클래스 번호이다.

3.1 패키지/컴포넌트 #1 : Parser

SQL 문을 토큰으로 나누고 구문을 분석하여 AST(Statement)를 만든다.

클래스명	역할 및 책임	주요기능 및 메서드	비고
Lexer	SQL 원문을 토큰으로 나눈다.	tokenize(), read_ident() / read_number() / read_string(), skip_whitespace()	C-001
Parser	토큰을 구문 분석하여 문장(AST)을 만든다.	parse(), parse_select() / parse_insert() / parse_update() / parse_delete(), parse_condexpr()	C-002
Statement	SQL 문장 종류별 AST와 열·외래키·조건식 정의를 담는 데이터 구조.		C-003

3.2 패키지/컴포넌트 #2 : Executor

AST를 실행한다(DDL, DML, 트랜잭션, 사용자, 프로시저, 트리거, 운영 문장). 세션 상태와 공유 DB 상태를 가진다.

클래스명	역할 및 책임	주요기능 및 메서드	비고
Executor	문장을 실행하고 세션의 트랜잭션과 현재 데이터베이스를 관리한다.	execute_sql(text), execute(Statement), exec_select() / exec_insert() / exec_update() / exec_delete() / exec_merge()	C-005
SharedDatabase	세션들이 함께 쓰는 표, 인덱스, 계정, 락, 캐시를 모은다.	verify_mysql_native_password(), ensure_default_user()	C-006
KillScope	문장이 실행되는 동안 그 스레드에 KILL 요청 상태를 걸고 끝나면 되돌린다.	KillScope() / ~KillScope()	C-021

3.3 패키지/컴포넌트 #3 : Planner 등

비용 기반으로 실행 계획을 고르고, 결과 캐시, 병렬 실행, 행 단위 락과 데드락 감지를 제공한다.

클래스명	역할 및 책임	주요기능 및 메서드	비고
Planner	접근 경로, 조인 알고리즘, 조인 순서를 정한다.	plan(table, condition, joins), choose_access(table, cond, pk), estimate_rows() / estimate_cost()	C-004
LockManager	행 락, Gap Lock을 관리하고 대기 그래프로 데드락을 찾는다.	acquire(table, pk, txn_id) / acquire_shared(), acquire_gap(), creates_cycle(from, to)	C-007
QueryResultCache	같은 문장의 결과를 LRU로 저장하고 표가 바뀌면 버린다.	get() / put(), invalidate_table()	C-008
ThreadPool	필터, 그룹, 정렬, 집계를 여러 스레드로 나누어 실행한다.	global(), parallel_for(n, fn)	C-009

3.4 패키지/컴포넌트 #4 : Catalog

표, 열, 제약, 파티션 메타데이터를 보관한다.

클래스명	역할 및 책임	주요기능 및 메서드	비고

Catalog / TableSchema	표 이름별 스키마(열, 기본키, CHECK, UNIQUE, 파티션)를 보관한다.	create_table() / drop_table(), get_table() / get_table_mut()	C-010

3.5 패키지/컴포넌트 #5 : Storage

인덱스(B+Tree, Hash, 복합), 페이지 캐시, 표 파일 입출력으로 데이터를 저장하고 읽는다.

클래스명	역할 및 책임	주요기능 및 메서드	비고
BPlusTree	키 순서로 값을 저장하는 B+Tree 인덱스.	insert(key, value) / remove(key), search(key), range_search() / scan_from() / scan_to()	C-011
HashIndex	같은 키의 행을 바로 찾는 해시 인덱스.	get(key), insert_row() / remove_row(), rebuild()	C-012
CompositeIndex	여러 열을 이어 만든 키의 B+Tree 인덱스.	search_exact(), rows_with_prefix(), insert_row() / remove_row() / rebuild()	C-013
BufferPool	표의 행을 LRU로 캐시하고 디스크와 주고받는다.	get_page(table, disk), write_page() / flush_all(), hit_rate()	C-014
DiskManager	표 파일과 스키마 파일을 읽고 쓴다.	save_table() / load_table(), save_schema() / load_schema(), list_databases()	C-015

3.6 패키지/컴포넌트 #6 : Transaction

MVCC 트랜잭션, redo 로그, 그룹 커밋, 크래시 복구를 담당한다.

클래스명	역할 및 책임	주요기능 및 메서드	비고
TransactionManager	트랜잭션 번호, 스냅샷, undo 기록, SERIALIZABLE 검증을 관리한다.	begin_with_snapshot(), commit() / rollback(), validate_serializable()	C-016
RedoLog	커밋한 변경을 체크섬이 붙은 묶음으로 덧붙이고 복구 때 읽는다.	append_batch(ops), sync(), read_all() / clear()	C-017
WalManager	변경 내용과 트랜잭션 결과를 WAL에 기록한다.	log_insert() / log_update() / log_delete(), log_commit() / log_rollback() / log_checkpoint(), needs_auto_checkpoint()	C-018
GroupCommitCoordinator	동시에 커밋하는 세션의 fsync를 한 번으로 묶는다.	sync_commit()	C-019

3.7 패키지/컴포넌트 #7 : Server · CLI · Client

클래스명	역할 및 책임	주요기능 및 메서드	비고
engine_server	자체 프로토콜(7878)과 MySQL 와이어 프로토콜(3306)로 접속을 받아 세션마다 Executor로 SQL을 실행한다.	handle_client(), start_mysql_listener(), wire_error()	C-020
engine_cli	터미널에서 SQL을 입력하는 REPL이다.	stdin 입력, 결과와 실행 시간 출력	
engine_client	자체 프로토콜(TCP)로 서버에 접속하는 클라이언트 라이브러리이다.	AUTH 챌린지-응답, SQL 전송, ---END--- 수신	

3.8 패키지/컴포넌트 #8 : Frontend (rusql-ui)

클래스명	역할 및 책임	주요기능 및 메서드	비고
App	홈 화면(연결 카드), SQL 편집기(Monaco), 결과 그리드, 서버 매니저를 담은 최상위 화면	연결 관리, 탭, 쿼리 실행, 서버 시작·중지	
Sidebar	데이터베이스, 표, 뷰, 인덱스 트리와 표 우클릭 메뉴	USE 전환, Select Rows, Describe, Import CSV	

ErdView, FkEditorModal	표 카드와 외래키 관계선, 외래키 편집기	카드 드래그, 열 끌어 연결, DDL 미리보기와 적용	
PlanDiagram, IndexAdvice	EXPLAIN 계획 그림과 인덱스 제안 카드	가장 느린 단계 표시, 임시 인덱스로 전·후 시간 비교	
MetricsPanel, ActivityPanel	서버 상태 추세 그래프, 세션·트랜잭션·락 패널	KILL, 느린 쿼리 로그, 복구 카드	
ResultChart, SchemaDiffModal, JourneyModal	결과 차트, 두 데이터베이스의 스키마 비교, 문장이 지나는 단계의 재생	막대·꺾은선·파이 차트, 마이그레이션 SQL	
DiagramView, BTreeView, MvccView	전체 구조 그림과 B+Tree, MVCC가 동작하는 모습을 스스로 재생하는 화면	한국어·영어 전환	

3.9 패키지/컴포넌트 #9 : MCP (rusql-mcp)

클래스명	역할 및 책임	주요기능 및 메서드	비고
mcp_server.py (데이터베이스 도구)	AI 어시스턴트가 SQL 실행, 스키마·인덱스·통계 조회, 실행 계획, 진단, 느린 쿼리 조회, 세션 종료, CSV 가져오기·내보내기, 스키마 비교를 하는 도구 20종	execute_sql(), run_script(), explain_query(), suggest_indexes(), health_report(), slow_queries(), kill_session(), import_csv(), diff_schema()	RuSQL 서버
mcp_server.py (연결·편집기 도구)	관리 도구의 저장된 연결과 편집기 탭을 다루는 도구 11종	list_connections(), write_to_editor(), execute_in_editor()	관리 도구
mcp_server.py (앱·서버 제어 도구)	앱 실행, 로그인, 서버 시작·중지·상태 확인 도구 6종	launch_app(), login(), start_server(), get_server_status()	관리 도구
위험 SQL 안전장치	DROP 등 되돌릴 수 없는 SQL은 확인 없이 실행하지 않는다.	confirm_dangerous_sql()	RuSQL 서버